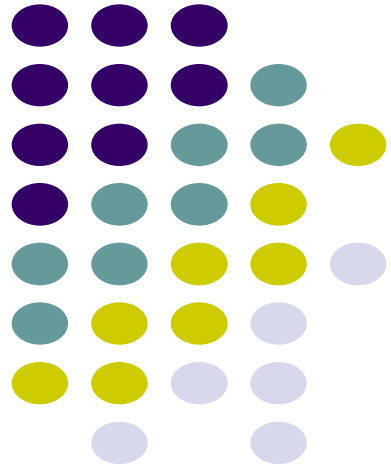
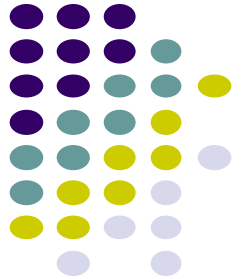
NP-Complete problems

Dr. Navjot Singh

Design and Analysis of Algorithms



Run-time analysis

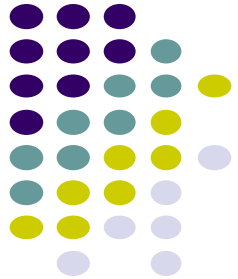


We've spent a lot of time in this class putting algorithms into specific run-time categories:

- O(log n)
- O(n)
- O(n log n)
- $O(n^2)$
- O(n log log n)
- $O(n^{1.67})$
- …

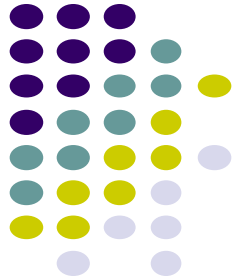
When I say an algorithm is O(f(n)), what does that mean?

Tractable vs. intractable problems



Tractable problems can be solved in O(f(n)) where f(n) is a polynomial

Tractable vs. intractable problems



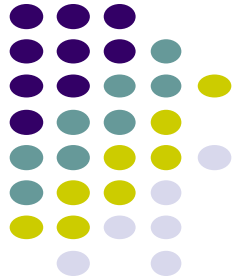
Tractable problems can be solved in O(f(n)) where f(n) is a polynomial

What about…

$O(n^{100})$?

$O(n^{\log \log \log \log n})$?

Tractable vs. intractable problems

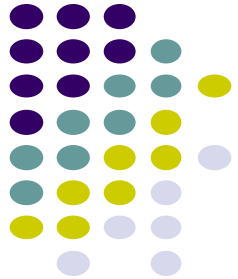


Tractable problems can be solved in O(f(n)) where f(n) is a polynomial

Technically $O(n^{100})$ is tractable by our definition

Why don't we worry about problems like this?

Tractable vs. intractable problems

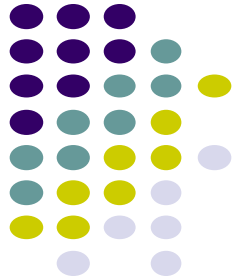


Tractable problems can be solved in O(f(n)) where f(n) is a polynomial

Technically $O(n^{100})$ is tractable by our definition

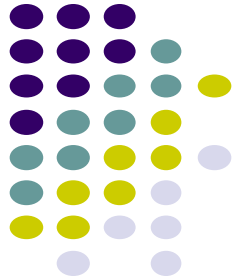
- Few practical problems result in solutions like this
- Once a polynomial time algorithm exists, more efficient algorithms are usually found
- Polynomial algorithms are amenable to parallel computation

Solvable vs. unsolvable problems



A problem is solvable if given enough (i.e. finite) time you could solve it

Sorting

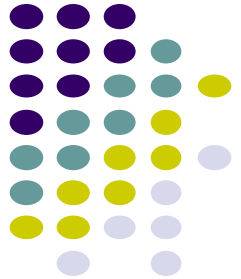


Given n integers, sort them from smallest to largest.

Tractable/intractable?

Solvable/unsolvable?

Sorting



Given n integers, sort them from smallest to largest.

Solvable and tractable:
Mergesort: $\Theta(n \log n)$

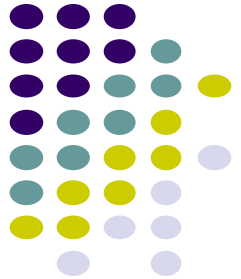
Enumerating all subsets

Given a set of n items, enumerate all possible subsets.

Tractable/intractable?

Solvable/unsolvable?

Enumerating all subsets

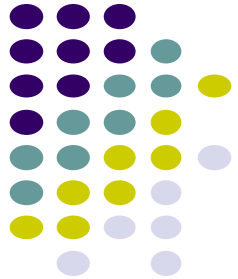


Given a set of n items, enumerate all possible subsets.

Solvable, but intractable: $\Theta(2^n)$ subsets

For large n this will take a very, very long time

Halting problem

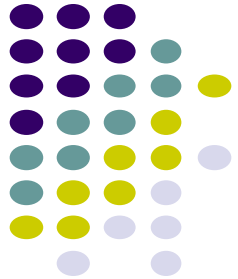


Given an arbitrary algorithm/program and a particular input, will the program terminate?

Tractable/intractable?

Solvable/unsolvable?

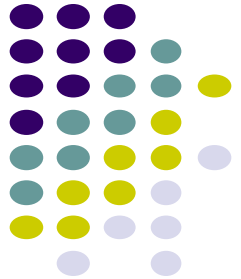
Halting problem



Given an arbitrary algorithm/program and a particular input, will the program terminate?

Unsolvable ☹

Integer solution?



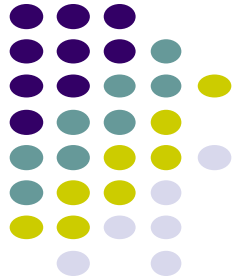
Given a polynomial equation, are there *integer* values of the variables such that the equation is true?

$$x^3yz + 2y^4z^2 - 7xy^5z = 6$$

Tractable/intractable?

Solvable/unsolvable?

Integer solution?



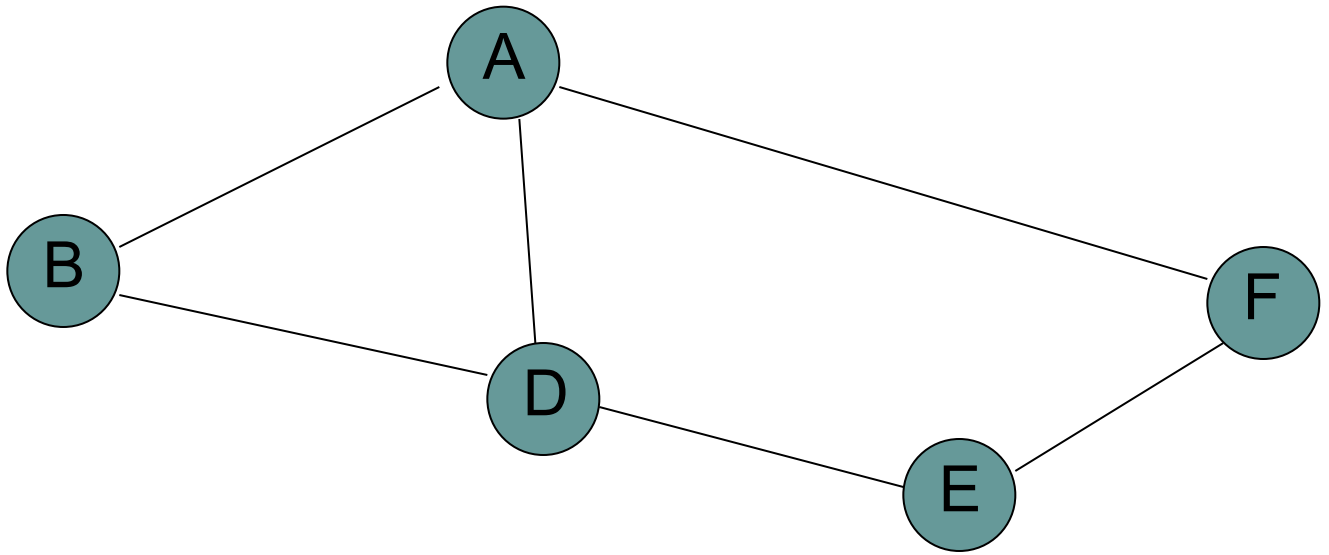
Given a polynomial equation, are there *integer* values of the variables such that the equation is true?

$$x^3yz + 2y^4z^2 - 7xy^5z = 6$$

Unsolvable ☹

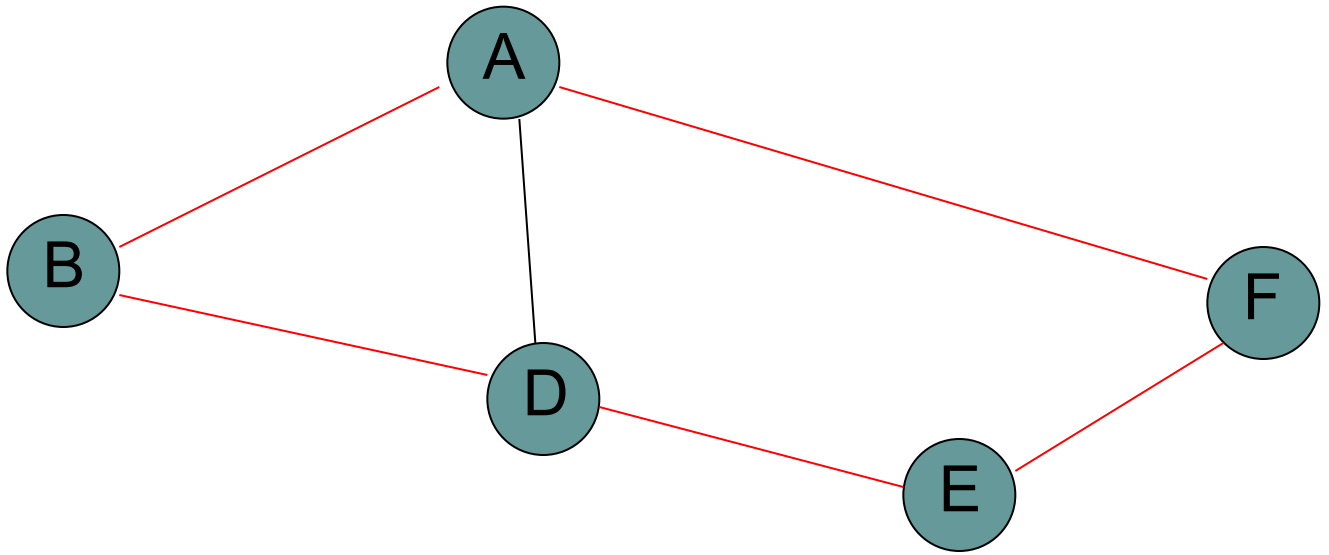
Hamiltonian cycle

Given an undirected graph G=(V, E), a hamiltonian cycle is a cycle that visits every vertex V exactly once



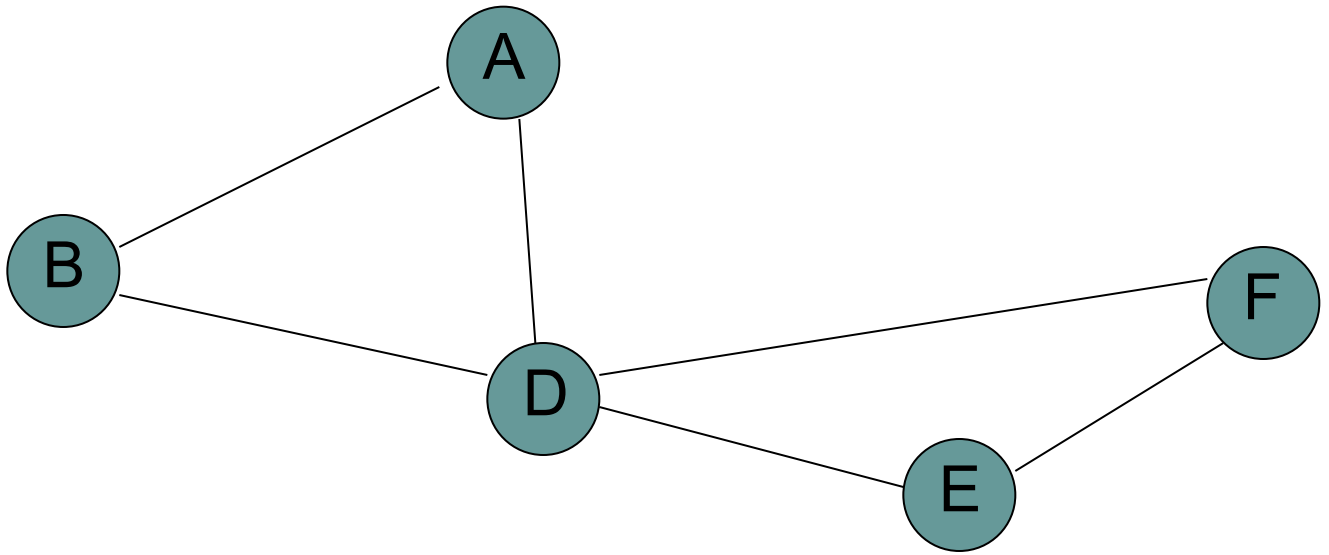
Hamiltonian cycle

Given an undirected graph G=(V, E), a hamiltonian cycle is a cycle that visits every vertex V exactly once



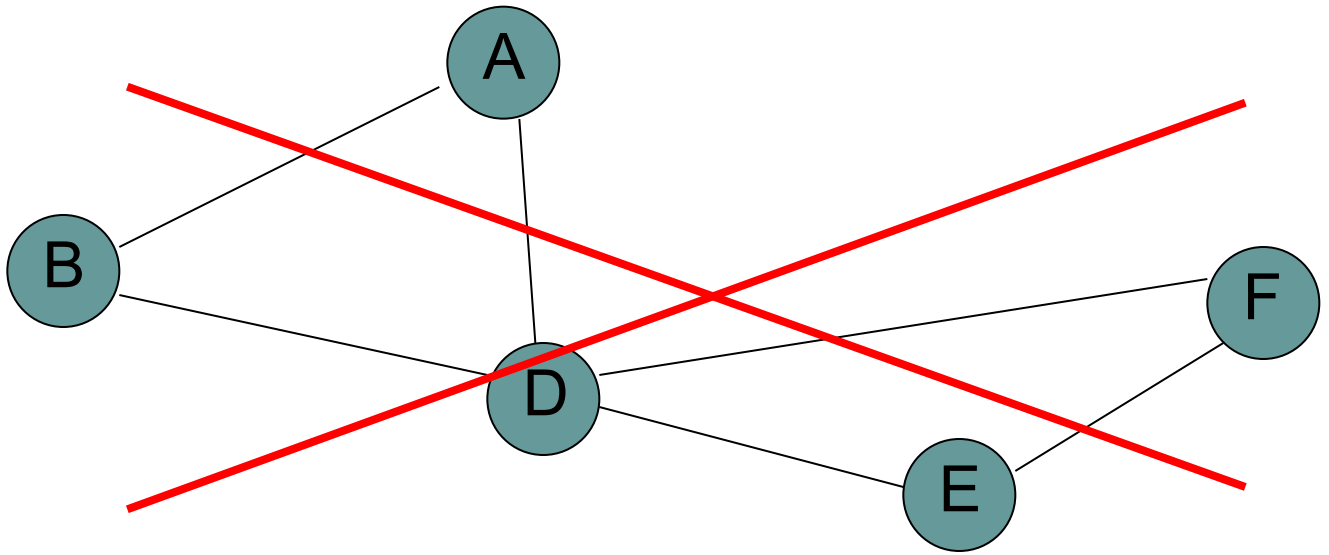
Hamiltonian cycle

Given an undirected graph G=(V, E), a hamiltonian cycle is a cycle that visits every vertex V exactly once

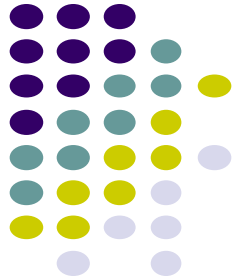


Hamiltonian cycle

Given an undirected graph G=(V, E), a hamiltonian cycle is a cycle that visits every vertex V exactly once



Hamiltonian cycle

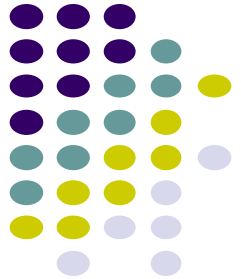


Given an undirected graph, does it contain a hamiltonian cycle?

Tractable/intractable?

Solvable/unsolvable?

Hamiltonian cycle



Given an undirected graph, does it contain a hamiltonian cycle?

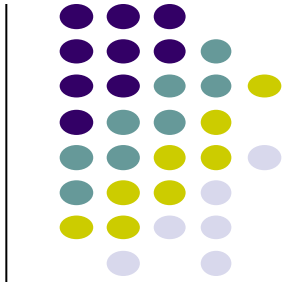
Solvable: Enumerate all possible paths (i.e. include an edge or don't) check if it's a hamiltonian cycle

How would we do this check exactly, specifically given a graph and a path?

Checking hamiltonian cycles

```
HAM-CYCLE-VERIFY(G, p)
 1  for i ← 1 to |V|
 2          visited[i] ← false
 3  n ← length[p]
 4  if p_1 ≠ p_n or n ≠ |V| + 1
 5          return false
 6  visited[p_1] ← true
 7  for i ← 1 to n − 1
 8          if visited[p_i]
 9                  return false
10          if (p_i, p_{i+1}) ∉ E
11                  return false
12          visited[p_i] ← true
13  for i ← 1 to |V|
14          if !visited[i]
15                  return false
16  return true
```

Checking hamiltonian cycles



```
HAM-CYCLE-VERIFY(G, p)
 1  for i ← 1 to |V|
 2          visited[i] ← false
 3  n ← length[p]
 4  if p_1 ≠ p_n or n ≠ |V| + 1
 5          return false
 6  visited[p_1] ← true
 7  for i ← 1 to n − 1
 8          if visited[p_i]
 9                  return false
10          if (p_i, p_{i+1}) ∉ E
11                  return false
12          visited[p_i] ← true
13  for i ← 1 to |V|
14          if !visited[i]
15                  return false
16  return true
```

Make sure the path starts and ends at the same vertex and is the right length

Can't revisit a vertex

Edge has to be in the graph

Check if we visited all the vertices

Checking hamiltonian cycles

```
HAM-CYCLE-VERIFY(G, p)
 1  for i ← 1 to |V|
 2          visited[i] ← false
 3  n ← length[p]
 4  if p_1 ≠ p_n or n ≠ |V| + 1
 5          return false
 6  visited[p_1] ← true
 7  for i ← 1 to n − 1
 8          if visited[p_i]
 9                  return false
10          if (p_i, p_{i+1}) ∉ E
11                  return false
12          visited[p_i] ← true
13  for i ← 1 to |V|
14          if !visited[i]
15                  return false
16  return true
```

Running time?

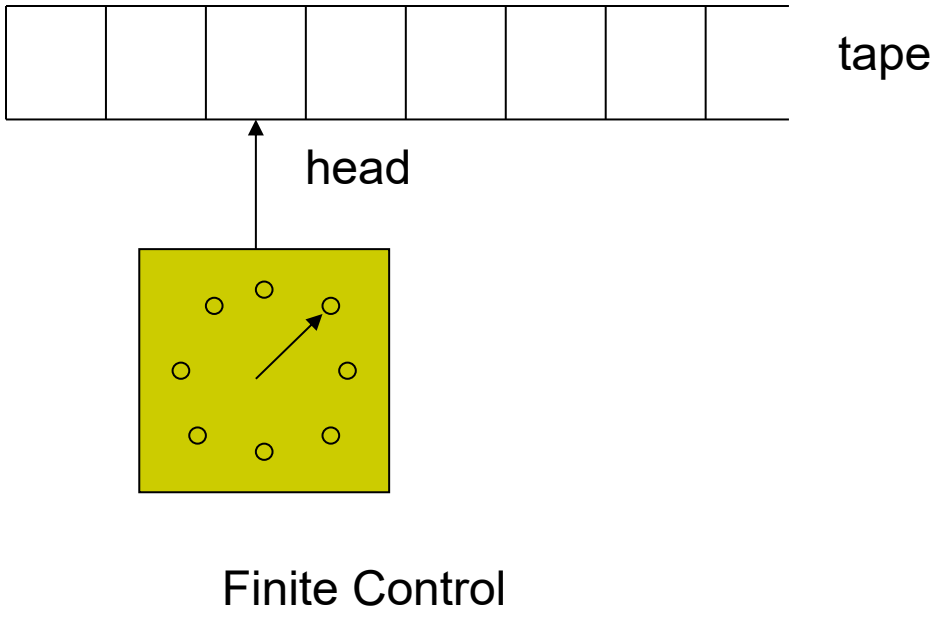
O(V) adjacency matrix

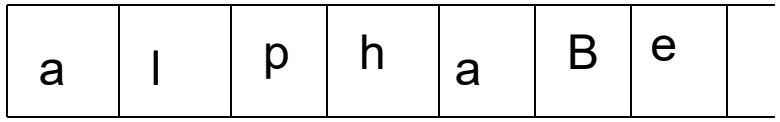
O(V+E) adjacency list

What does that say about the hamilonian cycle problem?

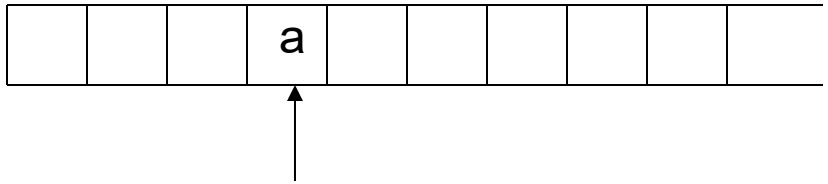
It is exponential time solvable

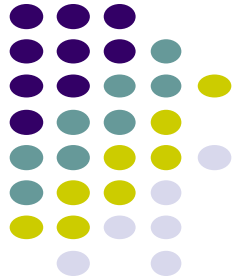
Deterministic Turing Machine (DTM)



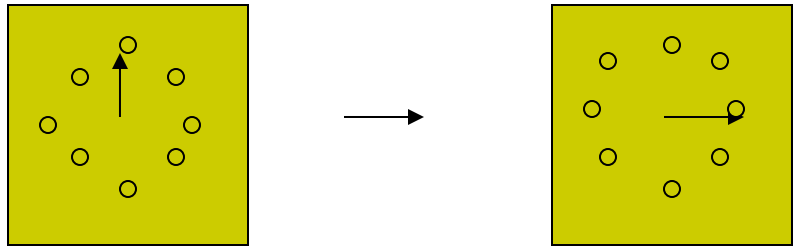


The tape has the left end but infinite to the right. It is divided into cells. Each cell contains a symbol in an alphabet Γ. There exists a special symbol B which represents the empty cell.



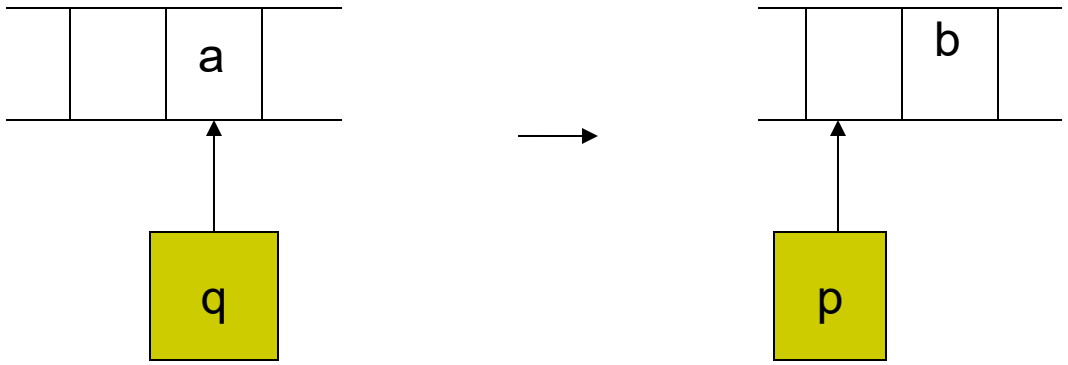


- The head scans at a cell on the tape and can *read*, *erase*, and *write* a symbol on the cell. In each move, the head can move to the right cell or to the left cell (or stay in the same cell).

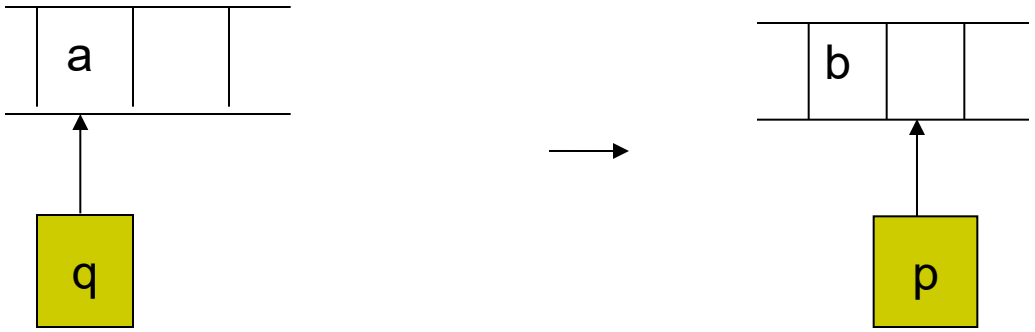


- The finite control has finitely many states which form a set Q. For each move, the state is changed according to the evaluation of a *transition function*

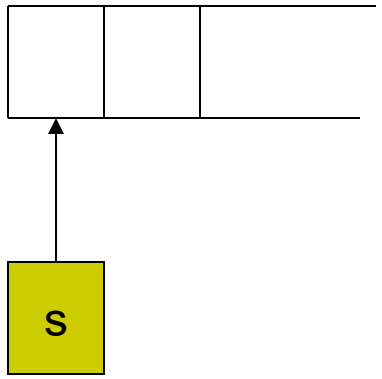
  $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{R, L\}$.



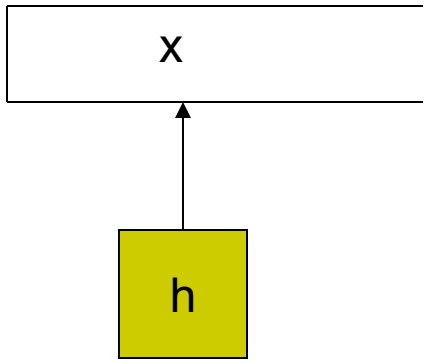
- $\delta(q, a) = (p, b, L)$ means that if the head reads symbol *a* and the finite control is in the state *q*, then the next state should be *p*, the symbol *a* should be changed to *b*, and the head moves one cell to the left.

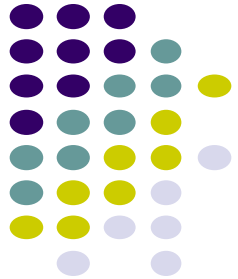


- $\delta(q, a) = (p, b, R)$ means that if the head reads symbol *a* and the finite control is in the state *q*, then the next state should be *p*, the symbol *a* should be changed to *b*, and the head moves one cell to the right.



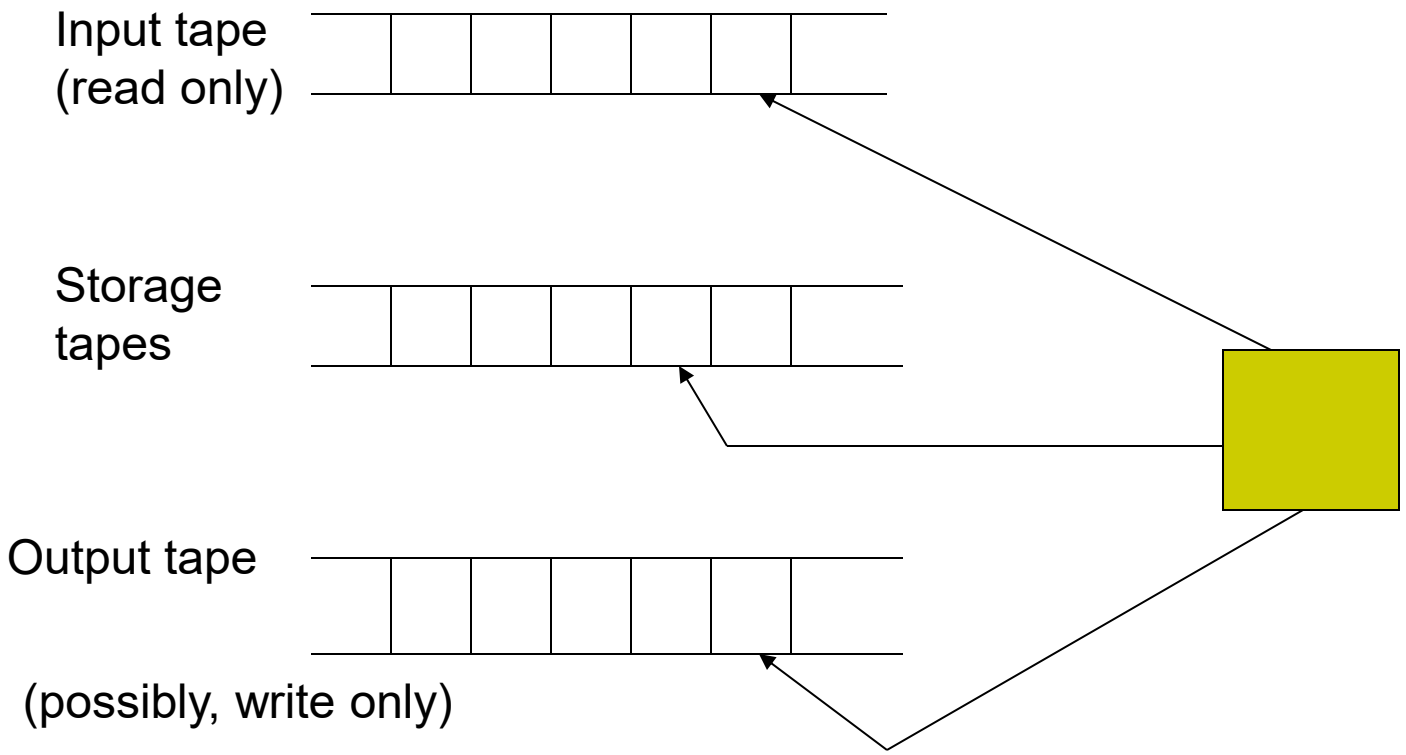
- There are some special states: an initial state *s* and and final states *h*.
- Initially, the DTM is in the initial state and the head scans the leftmost cell. The tape holds an input string.



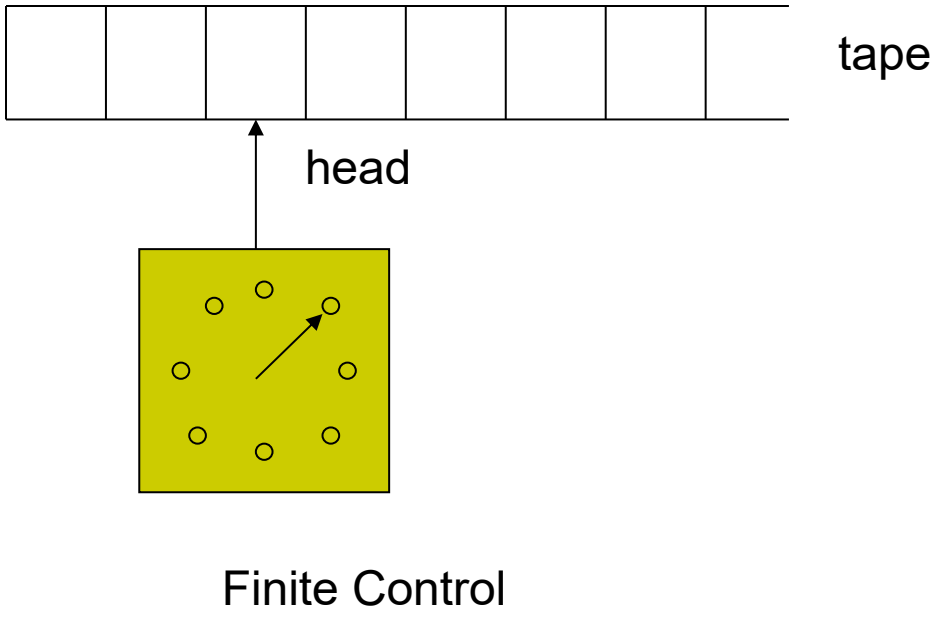


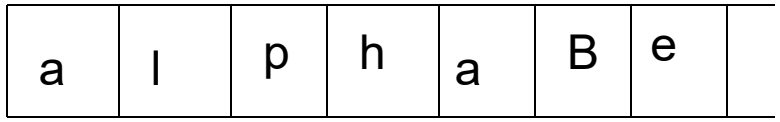
- When the DTM is in the final state, the DTM stops. An input string *x* is accepted by the DTM if the DTM reaches the final state *h*.
- Otherwise, the input string is rejected.

Multi-tape DTM

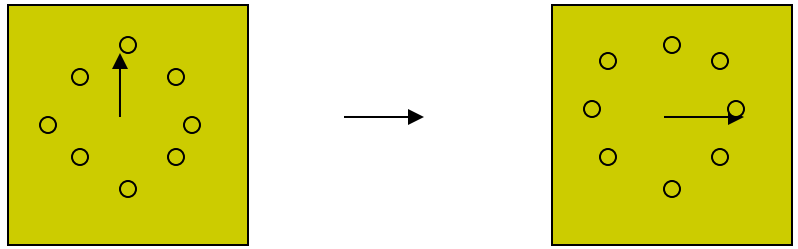


Nondeterministic Turing Machine (NTM)





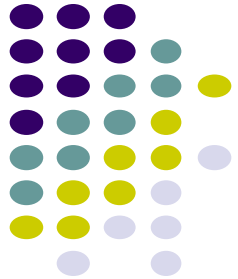
The tape has the left end but infinite to the right. It is divided into cells. Each cell contains a symbol in an alphabet Γ. There exists a special symbol B which represents the empty cell.



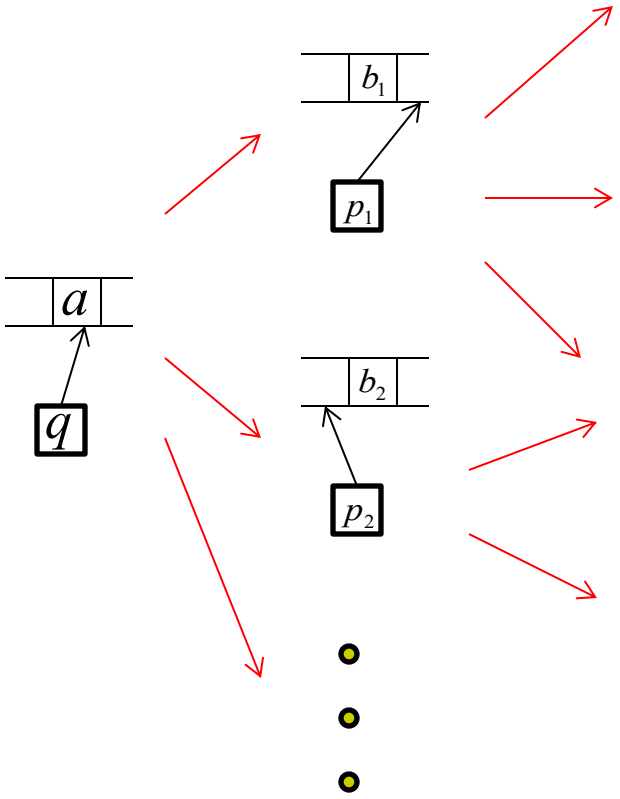
- The finite control has finitely many states which form a set Q. For each move, the state is changed according to the evaluation of a *transition function*

  $\delta : Q \times \Gamma \rightarrow 2^{\{Q \times \Gamma \times \{R, L\}\}}$.

Nondeterministic TM (NTM)



- There are multiple choices for each transition.
- For each input x, the NTM may have more than one computation paths.
- An input x is accepted if at least one computation path leads to the final state.
- L(M) is the set of all accepted inputs.



How many choices?

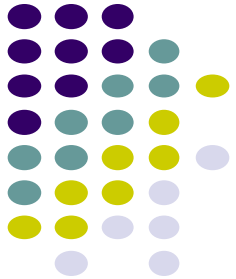
$$\leq |Q| \cdot |\Gamma| \cdot 3 = c$$

# of states

# of tape-symbols

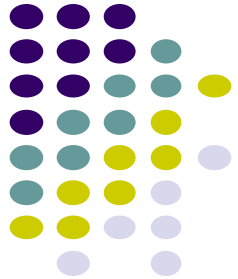
$$\delta(q,a) = \{(p_1, b_1, R), (p_2, b_2, L), \ldots\}$$

Time of DTM



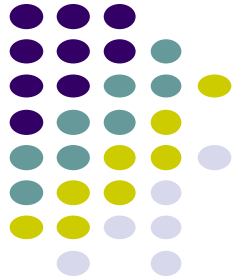
- $\text{Time}_M(x)$ = # of moves that DTM M takes on input x.
- $\text{Time}_M(x) < \text{infinity}$ iff $x \in L(M)$.

Space



- $\text{Space}_M(x)$ = maximum # of cells that M visits on each work (storage) tapes during the computation on input x.
- If M is a multitape DTM, then the work tapes do not include the input tape and the write-only output tape.

P problems

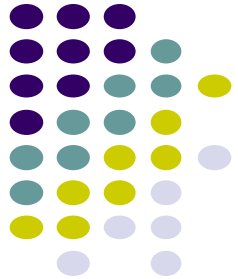


P = problems with a polynomial runtime solution

Also, called “tractable” problems

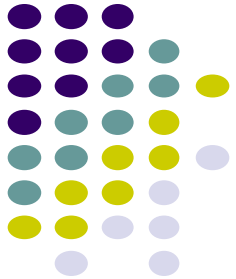
(Basically, all of the problems in this class)

NP problems



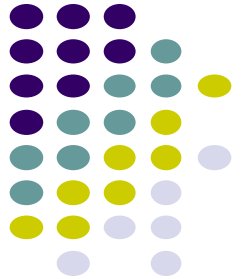
- NP is not the same as non-polynomial complexity/running time. NP does not stand for not polynomial.
- NP = Non-Deterministic polynomial time
- NP means verifiable in polynomial time
- Verifiable?
  - If we are somehow given a ‘certificate’ of a solution we can verify the legitimacy in polynomial time

What happened to automata?



- Problem is in NP iff it is decidable by some non deterministic Turing machine in polynomial time.
- Remember that the model we have used so far is a deterministic Turing machine
- It is provable that a Non Deterministic Turing Machine is equivalent to a Deterministic Turing Machine
- Remember NFA to DFA conversion?
  - Given an NFA with n states how many states does the equivalent DFA have?
  - Worst case …. 2^n
- The deterministic version of a poly time non deterministic Turing machine will run in exponential time (worst case)

NP problems

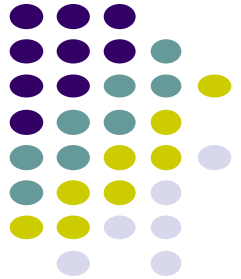


NP is the set of problems that can be *verified* in polynomial time

A problem can be verified in polynomial time if you can check that a given solution is correct in polynomial time

(NP is an abbreviation for non-deterministic polynomial time)

NP problems



Why might we care about NP problems?

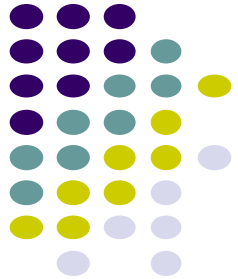
- If we can't verify the solution in polynomial time then an algorithm cannot exist that determines the solution in this time (why not?)
- All algorithms with polynomial time solutions are in NP

The NP problems that are currently not solvable in polynomial time *could in theory be solved in polynomial time*

P is a subset of NP

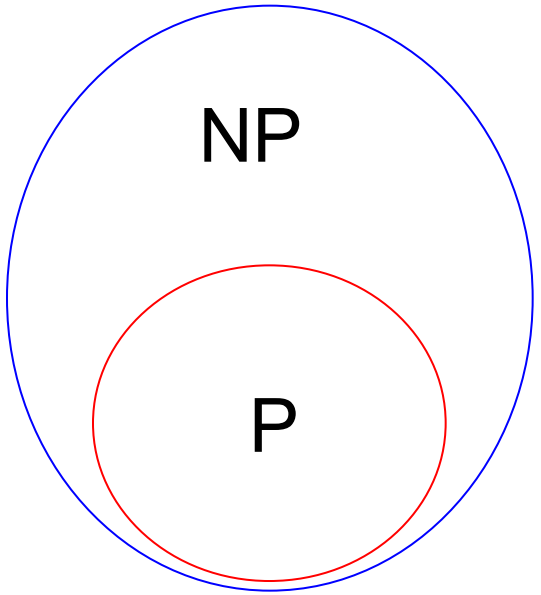
- Since it takes polynomial time to run the program, just run the program and get a solution
- But is NP a subset of P?
- No one knows if P = NP or not
- Solve for a million dollars!
  - http://www.claymath.org/millennium-problems
  - The Poincare conjecture is solved today

What is not in NP?



- Undecidable problems
  - Given a polynomial with integer coefficients, does it have integer roots
  - Hilbert's nth problem
  - Impossible to check for all the integers
  - Even a non-deterministic TM has to have a finite number of states!
  - More on decidability later
- Tautology
  - A boolean formula that is true for all possible assignments
  - Here just one 'verifier' will not work. You have to try all possible values

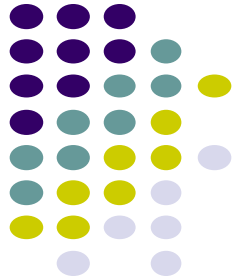
P and NP



Big-O allowed us to group algorithms by run-time

Today, we're talking about sets of problems grouped by how easy they are to solve

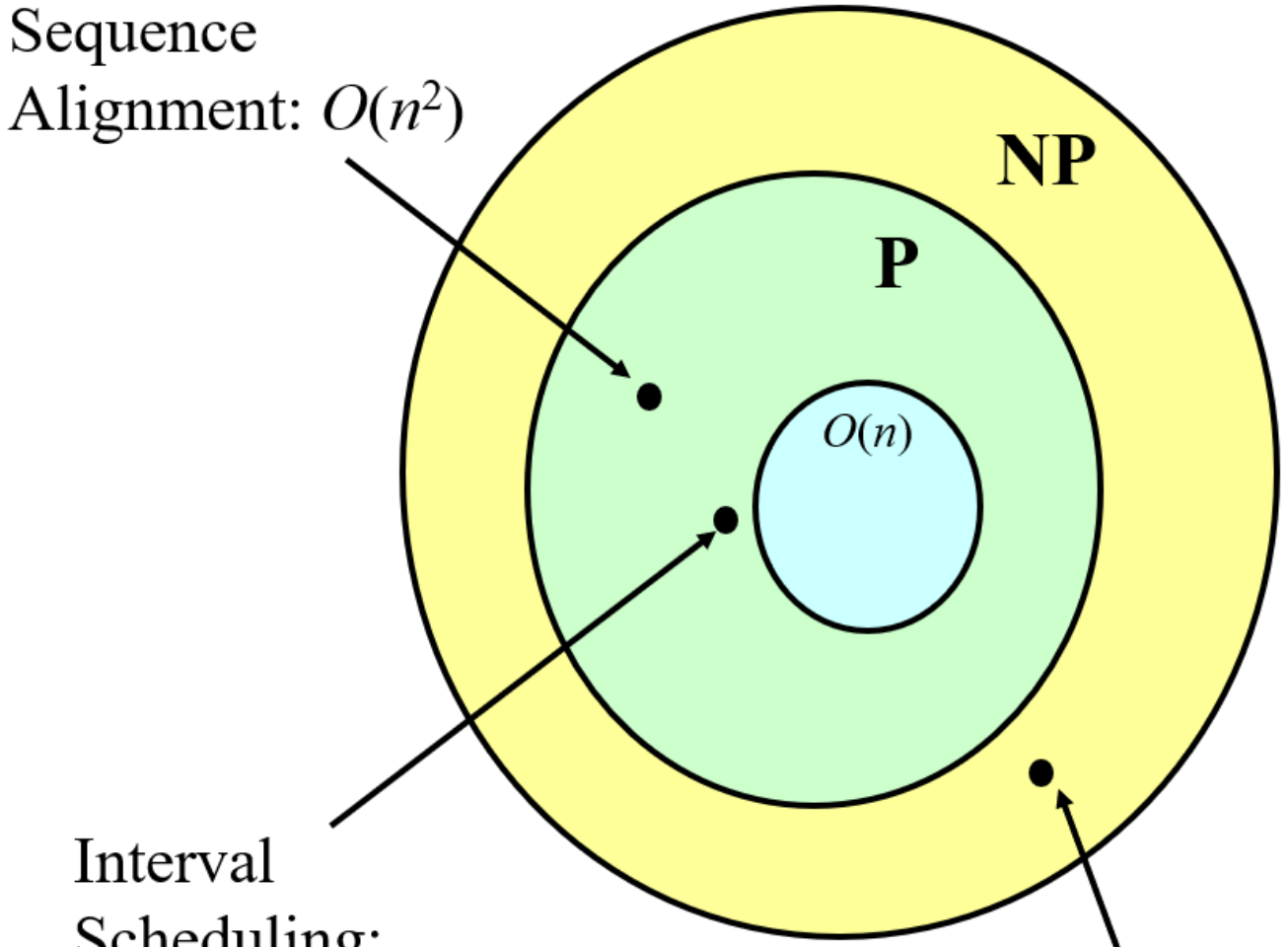
Complexity Classes



Class P: problems that can be solved in polynomial time ($O(n^k)$ for some constant k): "myopic" problems like sequence alignment, interval scheduling are all in **P**.

Class NP: problems that can be solved in polynomial time by a nondeterministic machine: includes all problems in **P** and some problems *possibly* outside **P** like subset sum.

Complexity Classes: Possible View



How many problems are in the $O(n)$ class?

infinite

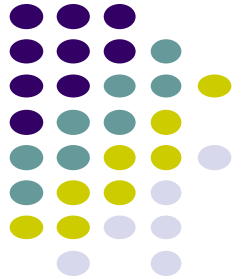
How many problems are in **P** but not in the $O(n)$ class?

infinite

How many problems are in **NP** but not in **P**?

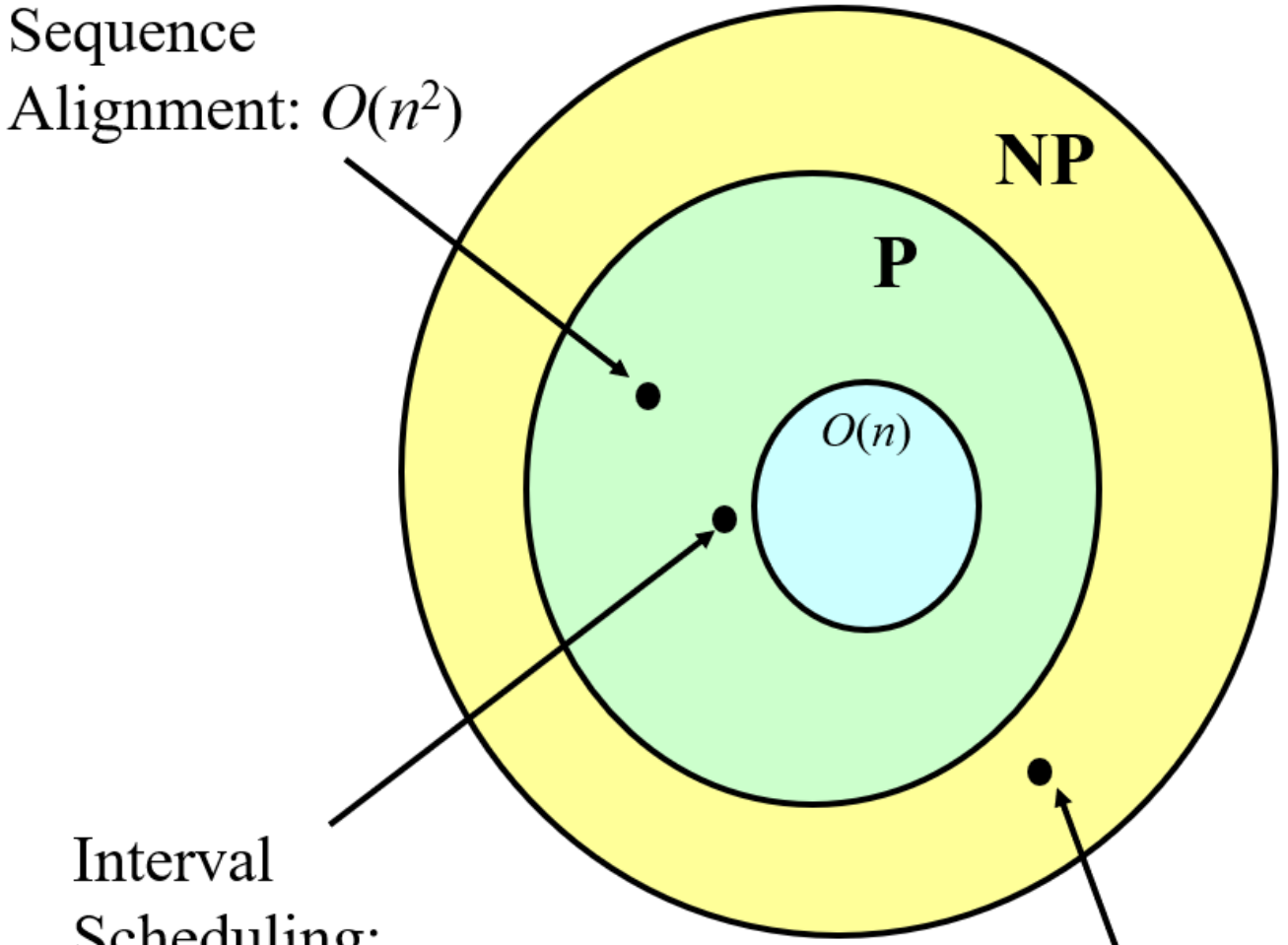
Either 0 or infinite!

P = NP?



- Is P different from NP: is there a problem in NP that is not also in P
  - If there is one, there are infinitely many
- Is the “hardest” problem in NP also in P
  - If it is, then every problem in NP is also in P
- No one knows the answer!
- The most famous unsolved problem in computer science and math
  - Listed first on Millennium Prize Problems

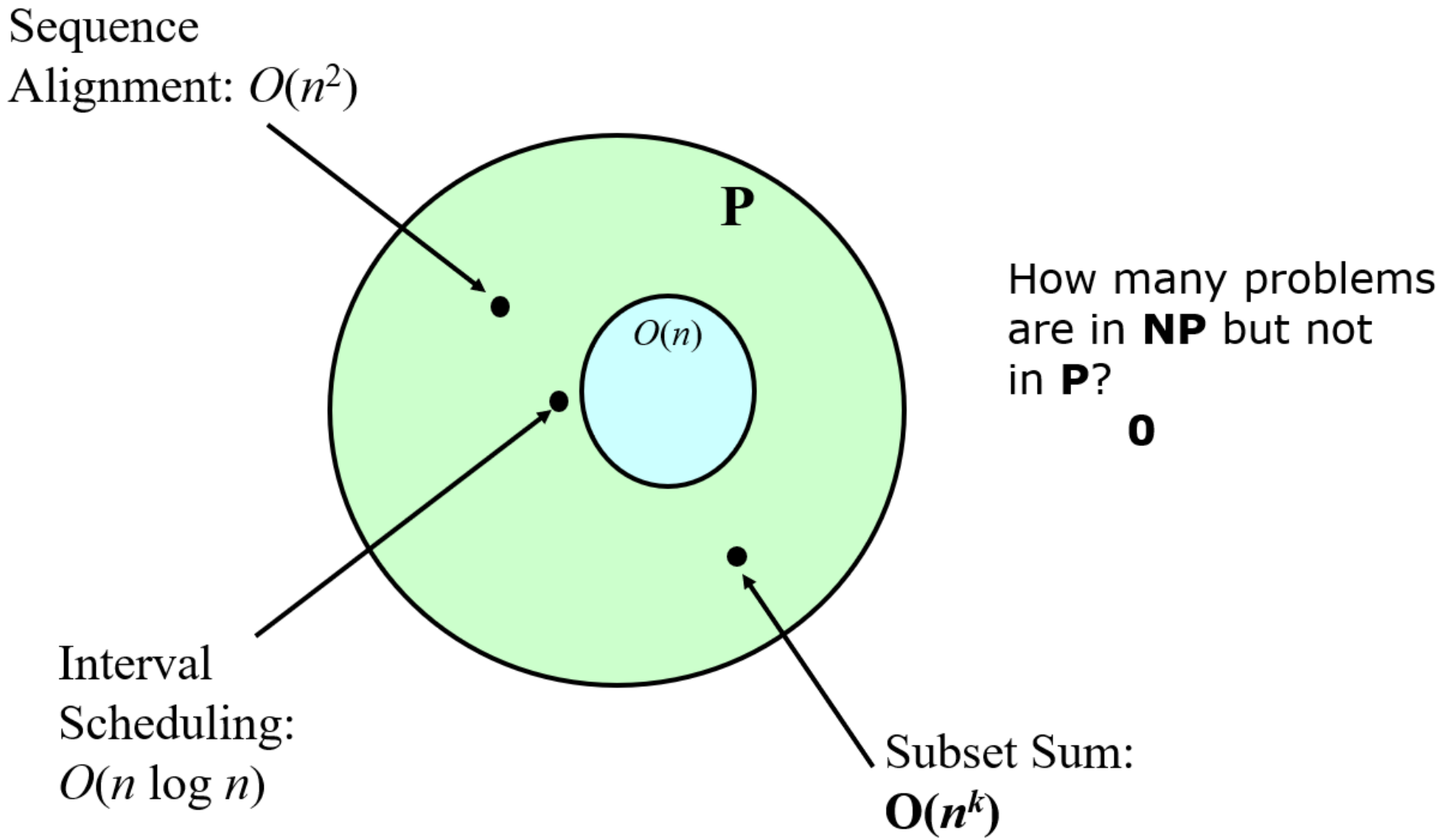
Problem Classes if P ⊂ NP:



How many problems are in **NP** but not in **P**?

Infinite!

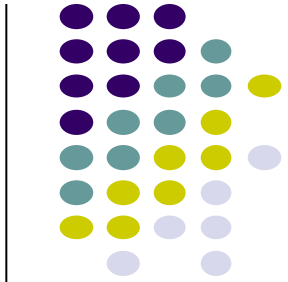
Problem Classes if P = NP:



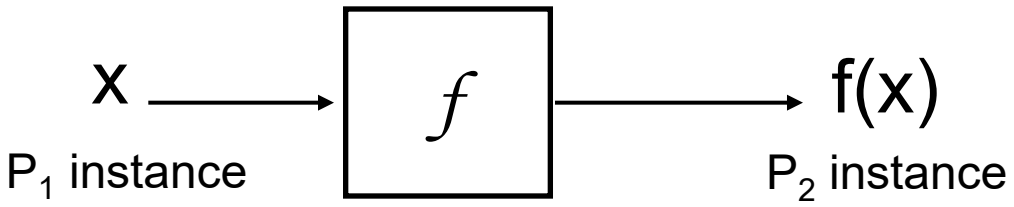
How many problems are in **NP** but not in **P**?

0

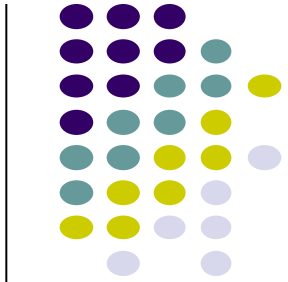
Reduction function



Given two problems P_1 and P_2 a *reduction function*, *f(x)*, is a function that transforms a problem instance *x* of type P_1 to a problem instance of type P_2 such that: a solution to *x* exists for P_1 iff a solution for *f(x)* exists for P_2

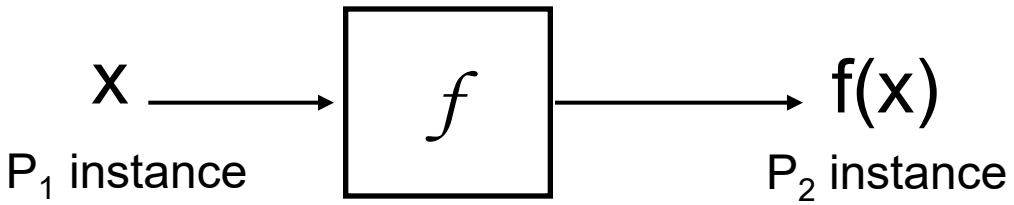


Reduction function

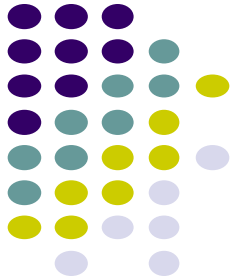


Given two problems P_1 and P_2 a *reduction function*, *f(x)*, is a function that transforms a problem instance *x* of type P_1 to a problem instance of type P_2 such that: a solution to *x* exists for P_1 iff a solution for *f(x)* exists for P_2

Is a linear equation reducible to a quadratic equation?



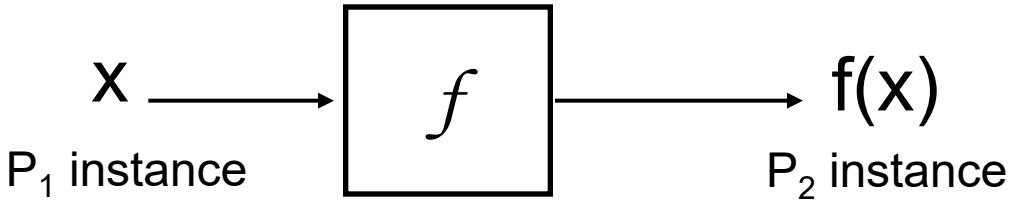
Reduction function



Given two problems P_1 and P_2 a *reduction function*, *f(x)*, is a function that transforms a problem instance *x* of type P_1 to a problem instance of type P_2 such that: a solution to *x* exists for P_1 iff a solution for *f(x)* exists for P_2

Is a linear equation reducible to a quadratic equation?

Sure! Let coefficient of the square term be 0

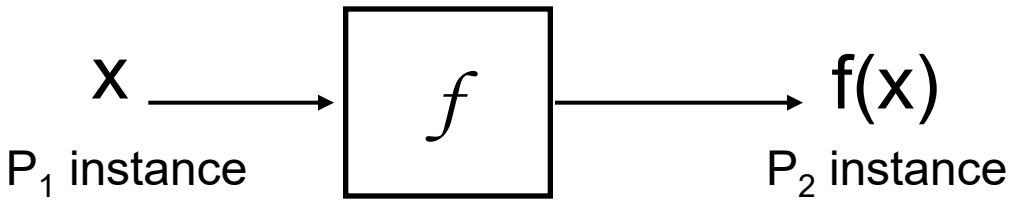


Reduction function

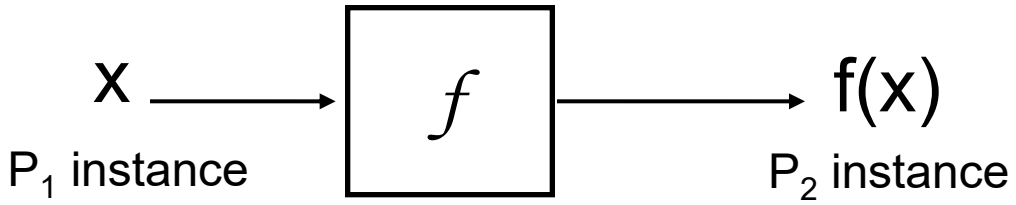
Where have we seen reductions before?

- Bipartite matching reduced to flow problem
- All pairs shortest path *through a particular vertex* reduced to single source shortest path

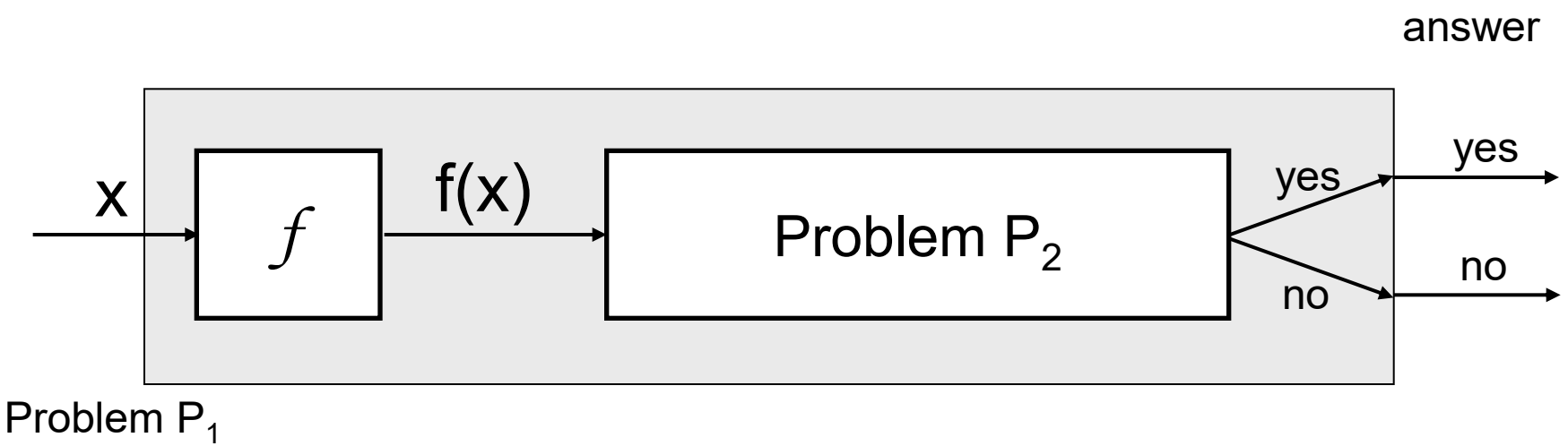
Why are they useful?



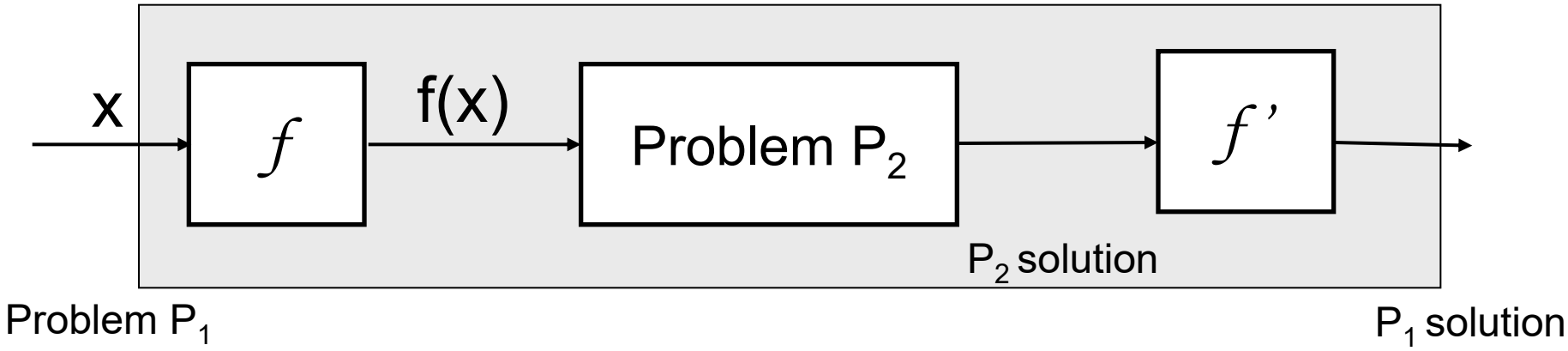
Reduction function



Allow us to solve P_1 problems if we have a solver for P_2

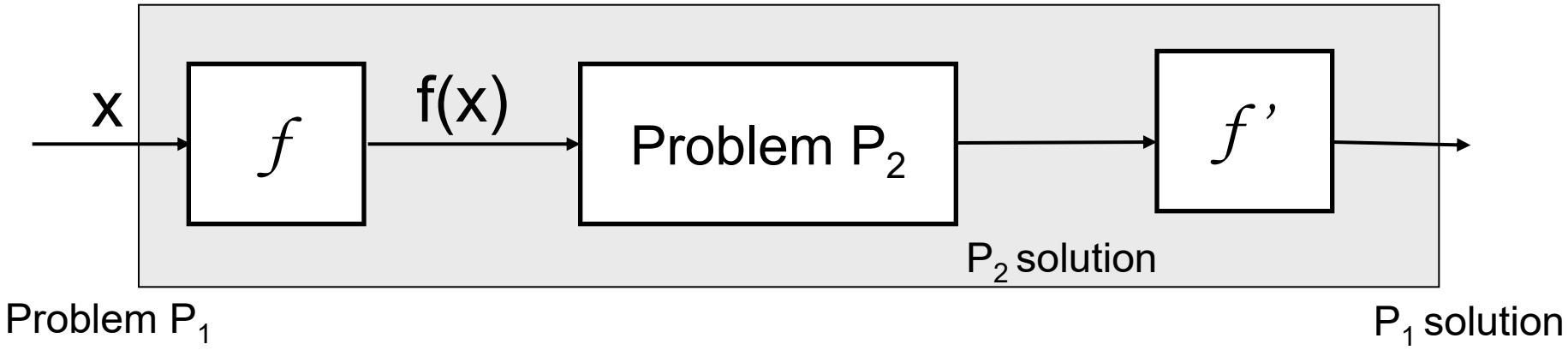


Reduction function



Most of the time we'll worry about yes no question, however, if we have more complicated answers we often just have to do a little work to the solution to the problem of P_2 to get the answer

Reduction function: Example

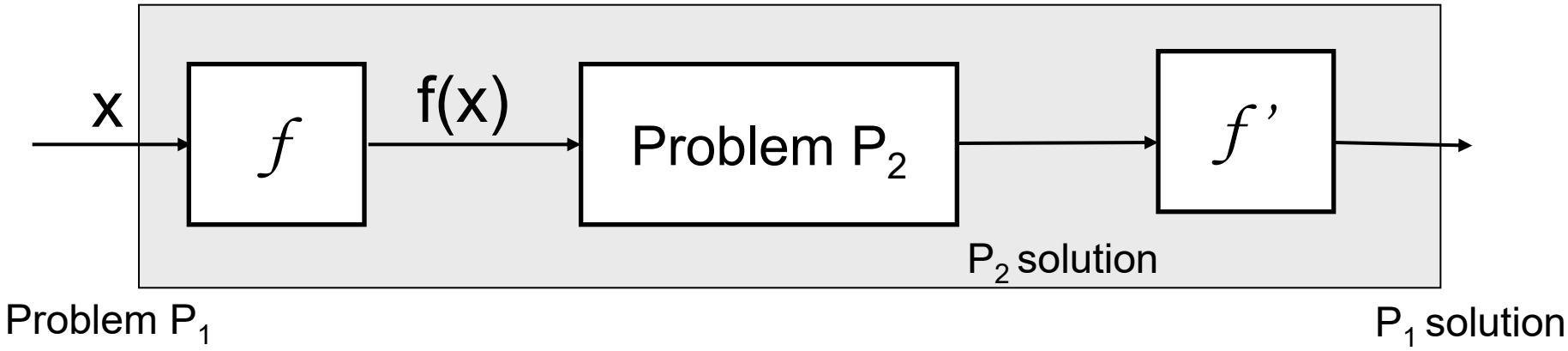


P1 = Bipartite matching
P2 = Network flow

Reduction function (f): Given *any* bipartite matching problem turn it into a network flow problem

What is *f* and what is *f'*?

Reduction function: Example

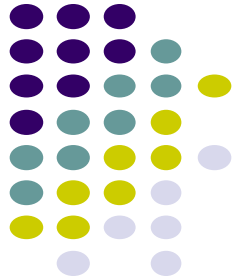


P1 = Bipartite matching
P2 = Network flow

Reduction function (f): Given *any* bipartite matching problem turn it into a network flow problem

A reduction function reduces problems instances

NP-Complete



A problem is *NP-complete* if:

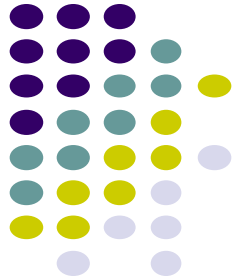
1. it can be verified in polynomial time (i.e. in NP)
2. *any* NP-complete problem can be reduced to the problem in polynomial time (is NP-hard)

A language $L \subseteq \{0,1\}^*$ is ***NP-complete*** if

1. $L \in \text{NP}$, and
2. $L' \leq_{\text{P}} L$ for every $L' \in \text{NP}$.

If a language L satisfies property 2, but not necessarily property 1, we say that L is ***NP-hard***. We also define NPC to be the class of NP-complete languages.

NP-Complete



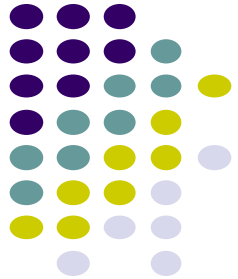
A problem is *NP-complete* if:

1. it can be verified in polynomial time (i.e. in NP)
2. *any* NP-complete problem can be reduced to the problem in polynomial time (is NP-hard)

The hamiltonian cycle problem is NP-complete

What are the implications of this?
What does this say about how hard the hamiltonian cycle problem is compared to other NP-complete problems?

NP-Complete



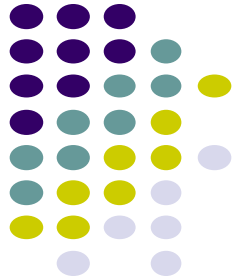
A problem is *NP-complete* if:

1. it can be verified in polynomial time (i.e. in NP)
2. *any* NP-complete problem can be reduced to the problem in polynomial time (is NP-hard)

The hamiltonian cycle problem is NP-complete

It's *at least as hard* as *any* of the other NP-complete problems

NP-Complete

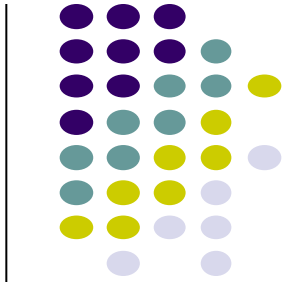


A problem is *NP-complete* if:

1. it can be verified in polynomial time (i.e. in NP)
2. *any* NP-complete problem can be reduced to the problem in polynomial time (is NP-hard)

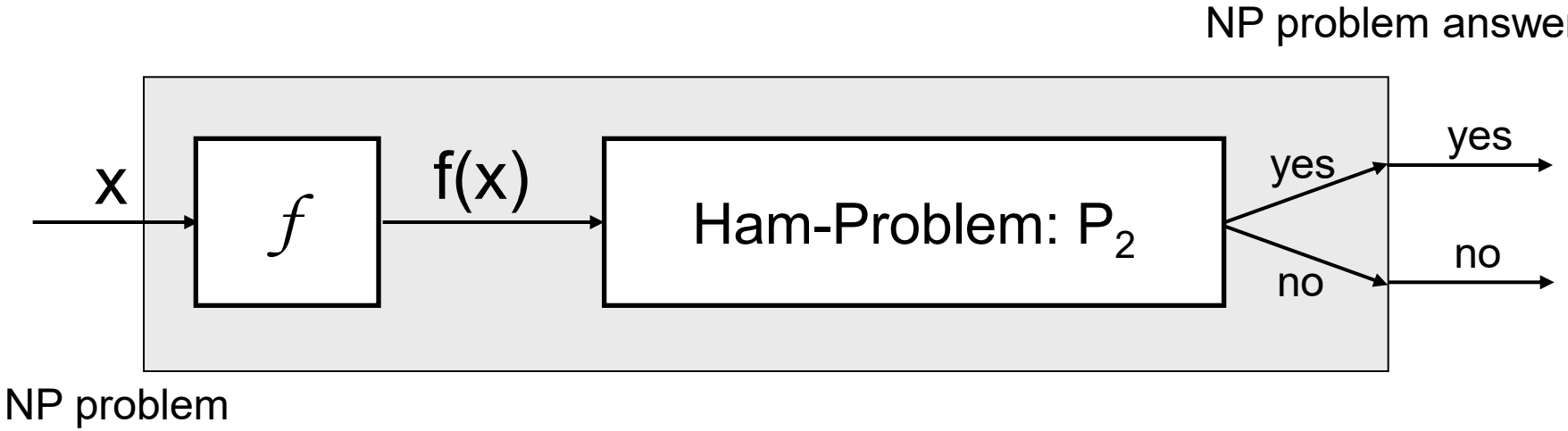
If I found a polynomial-time solution to the hamiltonian cycle problem, what would this mean for the other NP-complete problems?

NP-complete



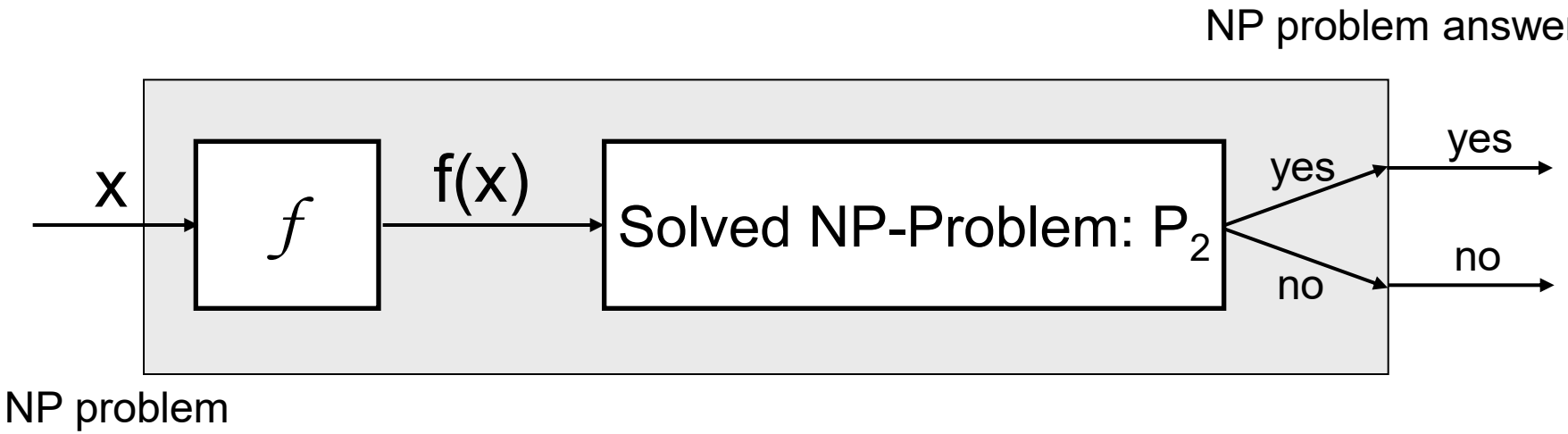
If a polynomial-time solution to the hamiltonian cycle problem is found, we would have a polynomial time solution to *any* NP-complete problem

- Take the input of the problem
- Convert it to the hamiltonian cycle problem (by definition, we know we can do this in polynomial time)
- Solve it
- If yes output yes, if no, output no

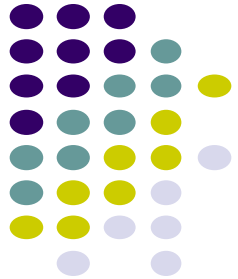


NP-complete

Similarly, if we found a polynomial time solution to *any* NP-complete problem we'd have a solution to *all* NP-complete problems



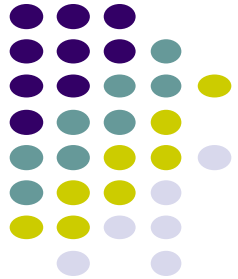
NP-complete problems



Longest path

Given a graph G with nonnegative edge weights does a simple path exist from *s* to *t* with weight at least *g*?

NP-complete problems



Longest path

Given a graph G with nonnegative edge weights does a simple path exist from *s* to *t* with weight at least *g*?

Integer linear programming

Linear programming with the constraint that the values must be integers

NP-complete problems

3D matching

Bipartite matching: given two sets of things and pair constraints, find a matching between the sets

3D matching: given three sets of things and triplet constraints, find a matching between the sets

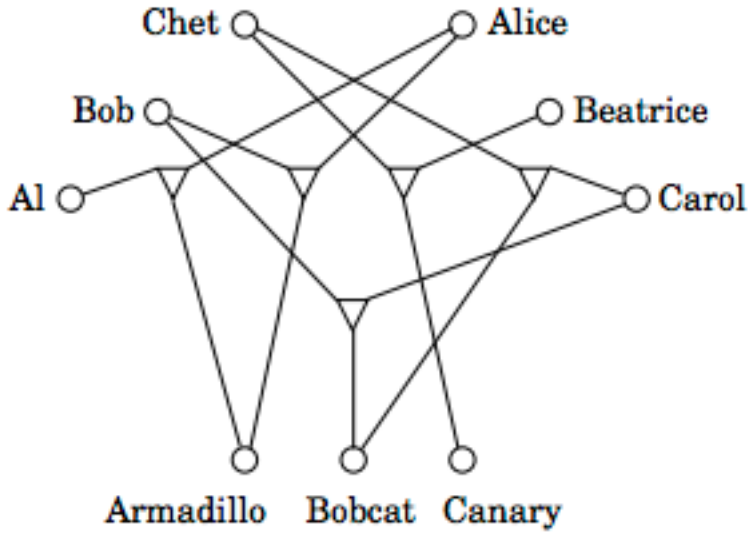
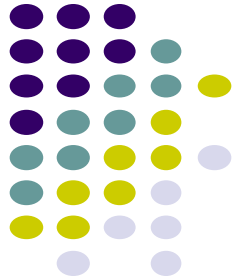


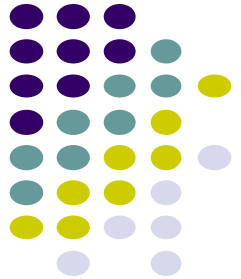
Figure from Dasgupta et. al 2008

P vs. NP



Polynomial time solutions exist	NP-complete (and no polynomial time solution currently exists)
Shortest path	Longest path
Bipartite matching	3D matching
Linear programming	Integer linear programming
Minimum cut	Balanced cut
…	…

Proving NP-completeness

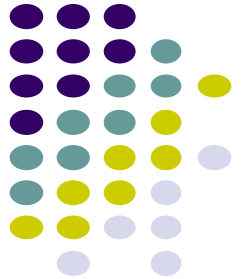


A problem is *NP-complete* if:

1. it can be verified in polynomial time (i.e. in NP)
2. *any* NP-complete problem can be reduced to the problem in polynomial time (is NP-hard)

Ideas?

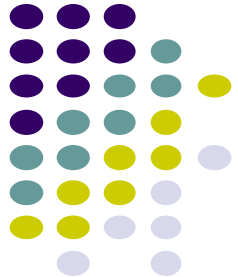
Proving NP-completeness



Given a problem NEW to show it is NP-Complete

1. Show that NEW is in NP
   a. Provide a verifier
   b. Show that the verifier runs in polynomial time
2. Show that all NP-complete problems are reducible to NEW in polynomial time
   a. Describe a reduction function *f* from a known NP-Complete problem to NEW
   b. Show that *f* runs in polynomial time
   c. Show that a solution exists to the NP-Complete problem IFF a solution exists *to the NEW problem generate by f*

Proving NP-completeness



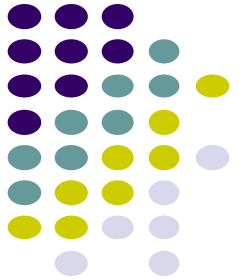
Show that a solution exists to the NP-Complete problem IFF a solution exists *to the NEW problem generate by f*

- Assume we have an NP-Complete problem instance that has a solution, show that the NEW problem instance generated by *f* has a solution

- Assume we have a problem instance of NEW *generated by f* that has a solution, show that we can derive a solution to the NP-Complete problem instance

Other ways of proving the IFF, but this is often the easiest

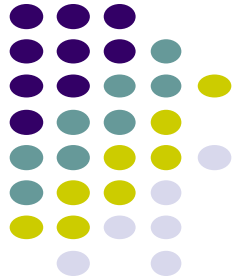
Proving NP-completeness



Show that all NP-complete problems are reducible to NEW in polynomial time

Why is it sufficient to show that one NP-complete problem reduces to the NEW problem?

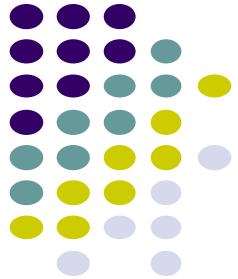
Proving NP-completeness



Show that all NP-complete problems are reducible to NEW in polynomial time

All others can be reduced to NEW by first reducing to the one problem, then reducing to NEW. Two polynomial time reductions is still polynomial time!

Proving NP-completeness



Show that all NP-complete problems are reducible to NEW in polynomial time

Show that *any* NP-complete problem is reducible to NEW in polynomial time

BE CAREFUL!

~~Show that NEW is reducible to any NP-complete problem in polynomial time~~

NP-complete: SAT

The (formula) satisfiability problem in terms of the language SAT is as follows. An instance of SAT is a Boolean formula Φ composed of

1. n boolean variables: $x_1 , x_2 , \ldots , x_n$;
2. m boolean connectives: any boolean function with one or two inputs and one output, such as ^ (AND), v (OR), ¬ (NOT), → (implication), ↔ (if and only if); and
3. parentheses. (Without loss of generality, assume that there are no redundant parentheses, i.e., a formula contains at most one pair of parentheses per Boolean connective.)

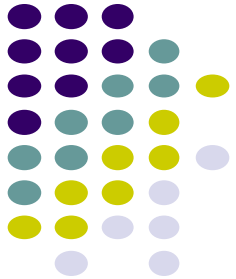
NP-complete: SAT

- We can encode a Boolean formula Φ with length polynomial in $n + m$.
- As in Boolean combinational circuits, a truth assignment for a Boolean formula Φ is a set of values for the variables of Φ, and a satisfying assignment is a truth assignment that causes it to evaluate to 1.
- A formula with a satisfying assignment is a satisfiable formula.
- The satisfiability problem asks whether a given Boolean formula is satisfiable, which we can express in formal-language terms as

$$SAT = \{ < \Phi > : \Phi \text{ is a satisfiable Boolean formula } \}$$

Is SAT an NP-complete problem?

NP-complete: SAT



As an example, the formula

$$\phi = ((x_1 \to x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

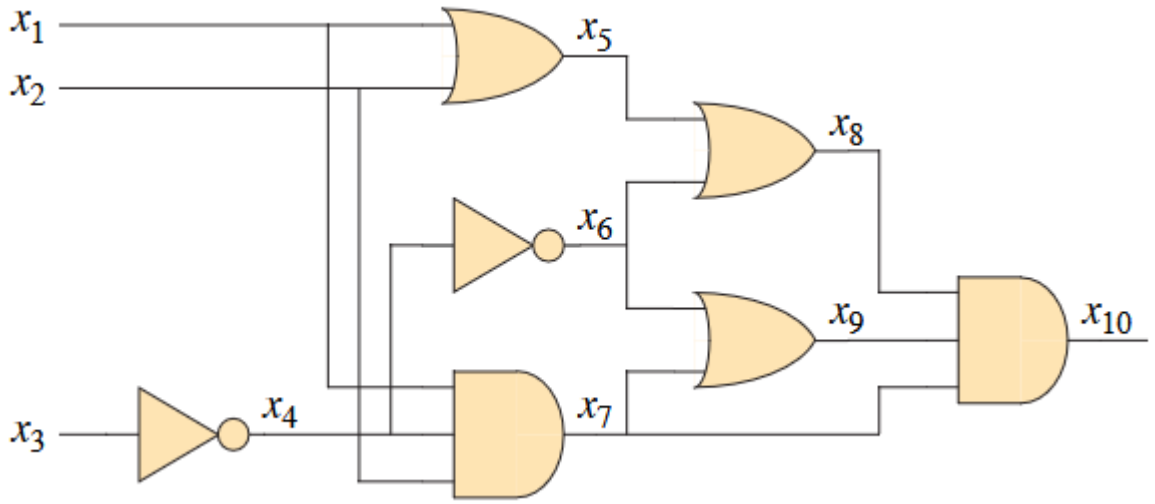
has the satisfying assignment $\langle x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1 \rangle$, since

$$\begin{aligned} \phi &= ((0 \to 0) \vee \neg((\neg 0 \leftrightarrow 1) \vee 1)) \wedge \neg 0 \\ &= (1 \vee \neg(1 \vee 1)) \wedge 1 \\ &= (1 \vee 0) \wedge 1 \\ &= 1 \,, \end{aligned}$$

The naive algorithm to determine whether an arbitrary Boolean formula is satisfiable does not run in polynomial time. A formula with n variables has 2^n possible assignments. If the length of $< \Phi >$ is polynomial in n, then checking every assignment requires $\Omega(2^n)$ time, which is superpolynomial in the length of $< \Phi >$.

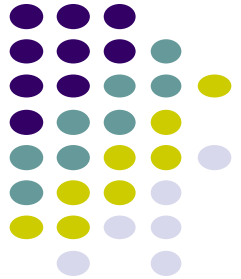
NP-complete: SAT

- To prove that SAT is NP-complete, we show that CIRCUIT-SAT $\leq_P$ SAT.



$$\begin{aligned}\phi = {} & x_{10} \wedge (x_4 \leftrightarrow \neg x_3) \\ & \wedge (x_5 \leftrightarrow (x_1 \vee x_2)) \\ & \wedge (x_6 \leftrightarrow \neg x_4) \\ & \wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_4)) \\ & \wedge (x_8 \leftrightarrow (x_5 \vee x_6)) \\ & \wedge (x_9 \leftrightarrow (x_6 \vee x_7)) \\ & \wedge (x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9))\end{aligned}$$

NP-complete: 3-CNF-SAT



A boolean formula is in *n-conjunctive normal form* (*n*-CNF) if:

- it is expressed as an AND of clauses
- where each clause is an OR of no more than *n* variables

$$(a \quad a \quad b) \ (c \quad b \quad d) \ (\ a \quad c \quad d)$$

3-CNF-SAT: Given a 3-CNF-SAT boolean formula, is it satisfiable?

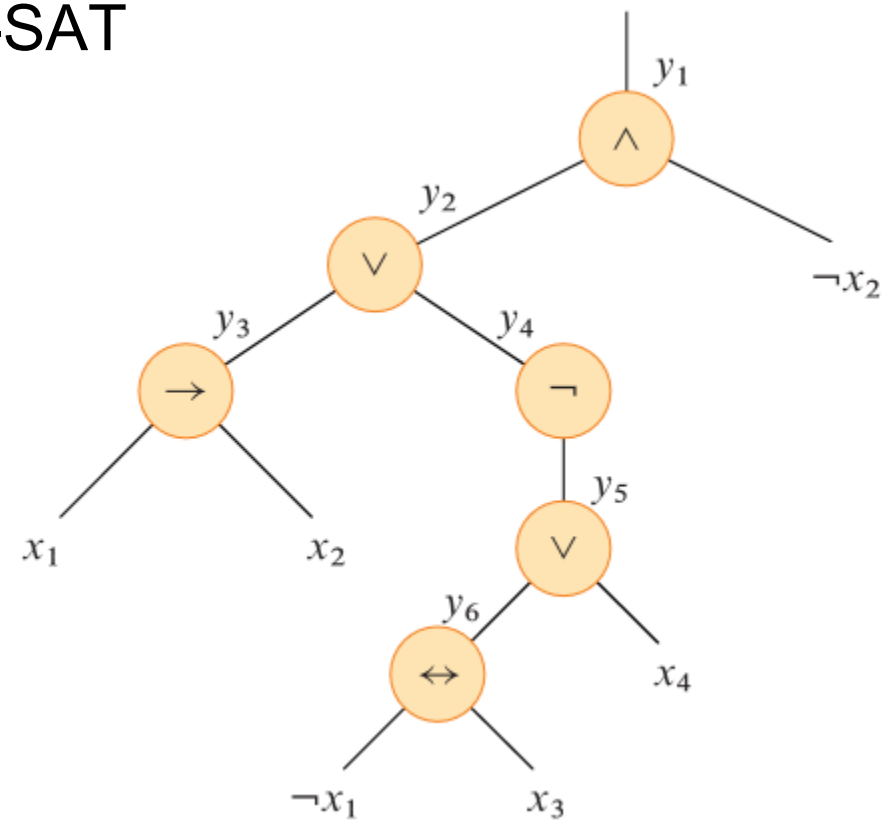
Is 3-CNF-SAT an NP-complete problem?

NP-complete: 3-CNF-SAT

SAT $\leq_P$ 3-CNF-SAT

$$\phi = ((x_1 \to x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

$$\begin{aligned}\phi' = \; & y_1 \\ & \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \\ & \wedge (y_2 \leftrightarrow (y_3 \vee y_4)) \\ & \wedge (y_3 \leftrightarrow (x_1 \to x_2)) \\ & \wedge (y_4 \leftrightarrow \neg y_5) \\ & \wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\ & \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3))\end{aligned}$$



NP-complete: 3-CNF-SAT

The truth table for the clause $(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$

y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	1

The Disjunctive Normal Form (DNF) formula equivalent to $\neg\phi'_1$ is

$$(y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2)$$

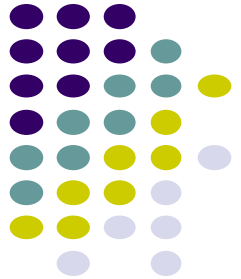
Negating and applying DeMorgan's laws yields the CNF formula

$$\begin{aligned}\phi''_1 = {} & (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \\ & \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2)\,,\end{aligned}$$

which is equivalent to the original clause ϕ'_1

The third and final step of the reduction further transforms the formula so that each clause has *exactly* three distinct literals. From the clauses of the CNF formula ϕ'', construct the final 3-CNF formula ϕ'''. This formula also uses two auxiliary variables, p and q. For each clause C_i of ϕ'', include the following clauses in ϕ''':

NP-complete: 3-CNF-SAT

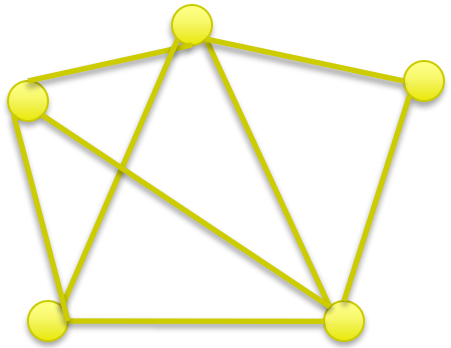


- If C_i contains three distinct literals, then simply include C_i as a clause of ϕ'''.
- If C_i contains exactly two distinct literals, that is, if $C_i = (l_1 \vee l_2)$, where l_1 and l_2 are literals, then include $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$ as clauses of ϕ'''. The literals p and $\neg p$ merely fulfill the syntactic requirement that each clause of ϕ''' contain exactly three distinct literals. Whether $p = 0$ or $p = 1$, one of the clauses is equivalent to $l_1 \vee l_2$, and the other evaluates to 1, which is the identity for AND.
- If C_i contains just one distinct literal l, then include $(l \vee p \vee q) \wedge (l \vee p \vee \neg q) \wedge (l \vee \neg p \vee q) \wedge (l \vee \neg p \vee \neg q)$ as clauses of ϕ'''. Regardless of the values of p and q, one of the four clauses is equivalent to l, and the other three evaluate to 1.

CLIQUE

A *clique* in an undirected graph G = (V, E) is a subset V' ⊆ V of vertices that are fully connected, i.e. every vertex in V' is connected to every other vertex in V'

CLIQUE problem: Does G contain a clique of size k?

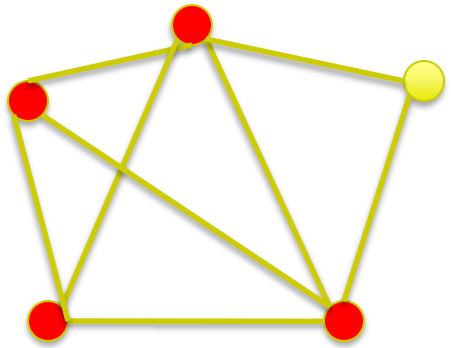


Is there a clique of size 4 in this graph?

CLIQUE

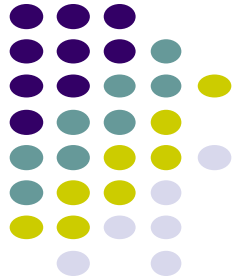
A *clique* in an undirected graph G = (V, E) is a subset V' ⊆ V of vertices that are fully connected, i.e. every vertex in V' is connected to every other vertex in V'

CLIQUE problem: Does G contain a clique of size k?



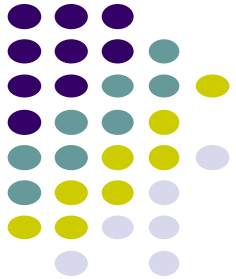
Is CLIQUE an NP-Complete problem?

CLIQUE



- First, we show that CLIQUE ϵ NP.
- For a given graph G = (V, E), use the set $V' \subseteq V$ of vertices in the clique as a certificate for G.
- To check whether V' is a clique in polynomial time, check whether, for each pair u, v ϵ V', the edge (u, v) belongs to E.

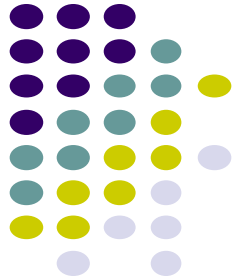
CLIQUE



$$3\text{-CNF-SAT} \leq_P \text{CLIQUE}$$

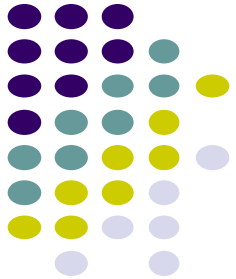
- Let $\Phi = C_1 \wedge C_2 \wedge \ldots \wedge C_k$ be a boolean formula in 3-CNF with k clauses.
- For r = 1, 2, …, k, each clause C_r contains exactly three distinct literals: l_1^r, l_2^r, l_3^r
- We will construct a graph G such that Φ is satisfiable if and only if G contains a clique of size k.

CLIQUE



- For each clause $C_r=(l_1^r \vee l_2^r \vee l_3^r)$ in Φ, place a triple of vertices v_1^r, v_2^r, v_3^r into V.
- Add edge (v_i^r, v_j^s) into E if both of the following hold:
  - v_i^r and v_j^s are in different triples, that is, $r \neq s$, and
  - their corresponding literals are consistent, that is, l_i^r is not the negation of l_j^s
- We can build this graph from Φ in polynomial time.

CLIQUE



$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$ $C_1 = x_1 \vee \neg x_2 \vee \neg x_3$

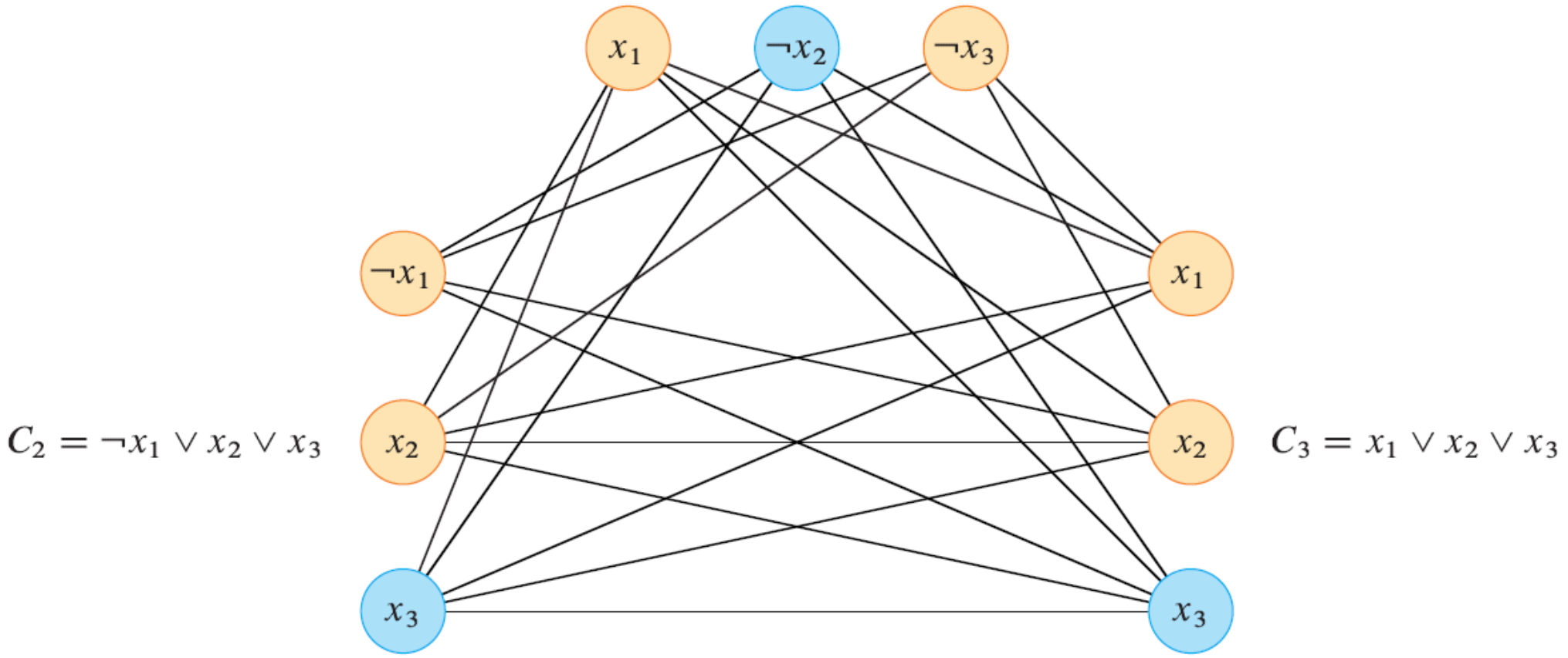
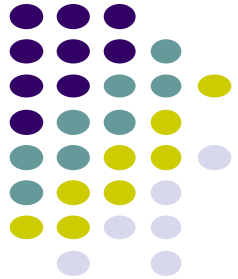


Figure 34.14 The graph G derived from the 3-CNF formula $\phi = C_1 \wedge C_2 \wedge C_3$, where $C_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$, $C_2 = (\neg x_1 \vee x_2 \vee x_3)$, and $C_3 = (x_1 \vee x_2 \vee x_3)$, in reducing 3-CNF-SAT to CLIQUE. A satisfying assignment of the formula has $x_2 = 0$, $x_3 = 1$, and x_1 set to either 0 or 1. This assignment satisfies C_1 with $\neg x_2$, and it satisfies C_2 and C_3 with x_3, corresponding to the clique with blue vertices.

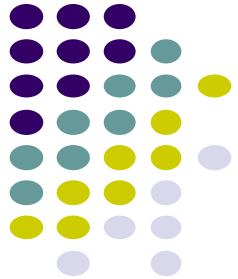
Vertex-cover problem



- Vertex cover: given an undirected graph G=(V,E), then a subset V'$\subseteq$V such that if (u,v)$\in$E, then $u\in$V' or $v\in$V' (or both).
- Size of a vertex cover: the number of vertices in it.
- Vertex-cover problem: find a vertex-cover of minimal size.

VERTEX-COVER = {<G,k>: graph G has a vertex cover of size k}

Vertex-cover problem



- First, we show that VERTEX-COVER ϵ NP.
- For a given graph G = (V, E) and an integer k, the certificate is the vertex cover V' $\subseteq$ V itself.
- The verification algorithm affirms that |V'| =k, and then it checks, for each edge (u, v) ϵ E, that u ϵ V' or v ϵ V'.
- It is easy to verify the certificate in polynomial time.
- Now CLIQUE $\leq_P$ VERTEX-COVER

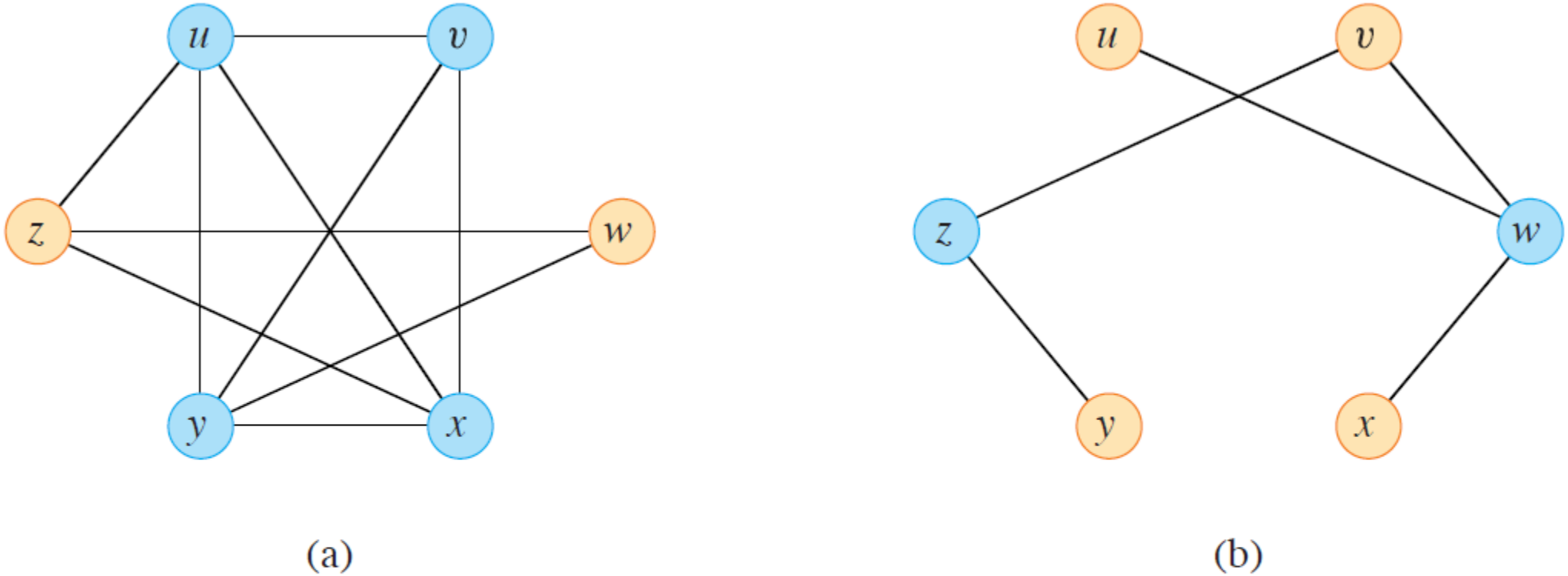
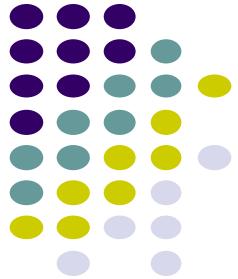


Figure 34.15 Reducing CLIQUE to VERTEX-COVER. **(a)** An undirected graph $G = (V, E)$ with clique $V' = \{u, v, x, y\}$, shown in blue. **(b)** The graph $\overline{G}$ produced by the reduction algorithm that has vertex cover $V - V' = \{w, z\}$, in blue.

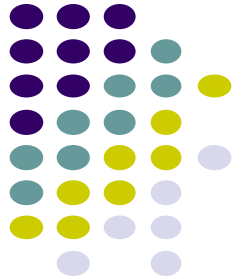
HALF-CLIQUE



Given a graph G, does the graph contain a clique containing exactly half the vertices?

Is HALF-CLIQUE an NP-complete problem?

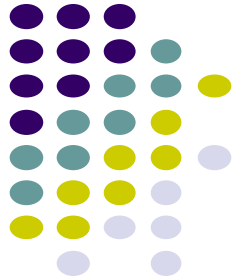
Is Half-Clique NP-Complete?



1. Show that NEW is in NP
   a. Provide a verifier
   b. Show that the verifier runs in polynomial time
2. Show that all NP-complete problems are reducible to NEW in polynomial time
   a. Describe a reduction function *f* from a known NP-Complete problem to NEW
   b. Show that *f* runs in polynomial time
   c. Show that a solution exists to the NP-Complete problem IFF a solution exists *to the NEW problem generate by f*

Given a graph G, does the graph contain a clique containing exactly half the vertices?

HALF-CLIQUE

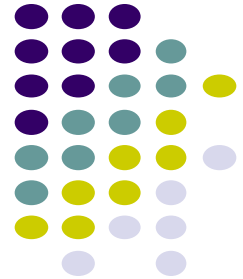


1. Show that HALF-CLIQUE is in NP
   a. Provide a verifier
   b. Show that the verifier runs in polynomial time

Verifier: A solution consists of the set of vertices in V'
•check that |V'| = |V|/2
•for all pairs of u, v ∈ V'
  • there exists an edge (u,v) ∈ E

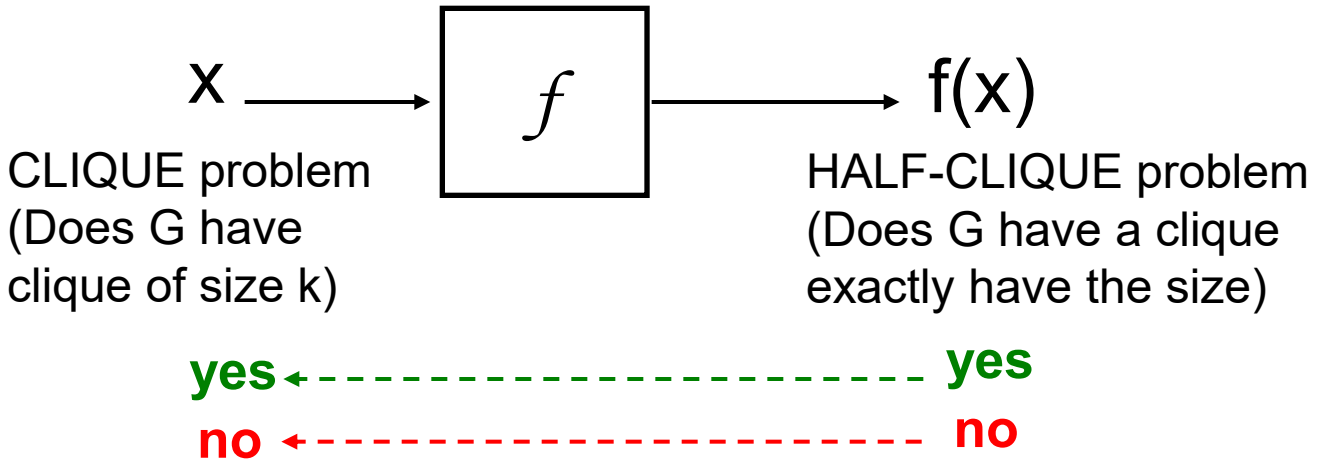
- Check for edge existence in O(V)
- $O(V^2)$ checks
- $O(V^3)$ overall, which is polynomial

HALF-CLIQUE

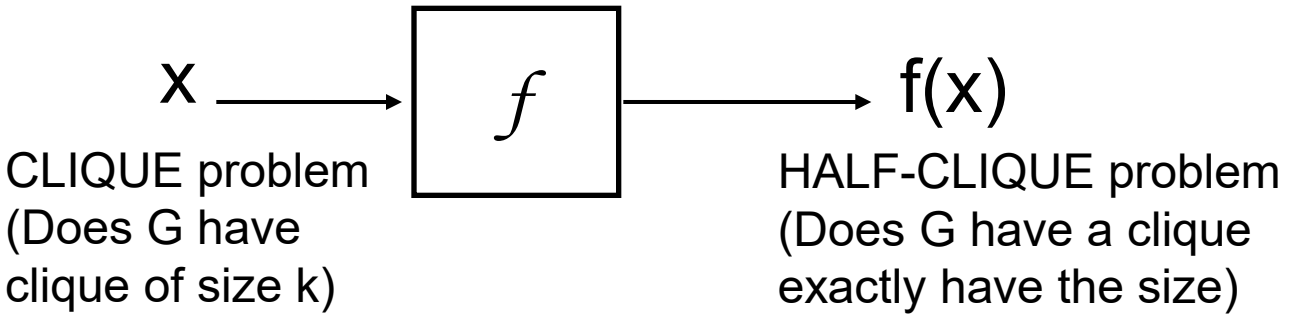


1.
2. Show that all NP-complete problems are reducible to HALF-CLIQUE in polynomial time
   a. Describe a reduction function *f* from a known NP-Complete problem to HALF-CLIQUE
   b. Show that *f* runs in polynomial time
   c. Show that a solution exists to the NP-Complete problem IFF a solution exists *to the HALF-CLIQUE problem generate by f*

Reduce CLIQUE to HALF-CLIQUE:
Given a problem instance of CLIQUE, turn it into a problem instance of HALF-CLIQUE



HALF-CLIQUE



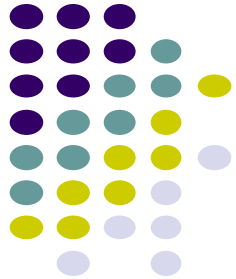
Three cases:

1. $k = |V|/2$

2. $k < |V|/2$

3. $k > |V|/2$

HALF-CLIQUE



Reduce CLIQUE to HALF-CLIQUE:
Given an instance of CLIQUE, turn it into an instance of HALF-CLIQUE

It's already a half-clique problem

$f(G, k)$

1 **if** $\lceil |V| \rceil / 2 = k$
2 **return** G
3 **elseif** $k < \lceil |V| \rceil / 2$
4 **return** G plus $(|V| - 2k)$ nodes which are fully connected and are connected to every node in V
5 **else**
6 **return** G plus $2k - |V|$ nodes which have no edges

HALF-CLIQUE

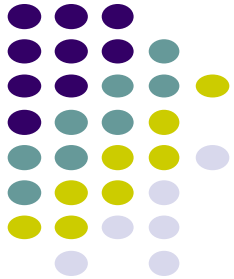
Reduce CLIQUE to HALF-CLIQUE:
Given an instance of CLIQUE, turn it into an instance of HALF-CLIQUE

We're looking for a clique that is smaller than half, so add an artificial clique to the graph and connect it up to all vertices

$f(G, k)$

1 **if** $\lceil |V| \rceil / 2 = k$
2 **return** G
3 **elseif** $k < \lceil |V| \rceil / 2$
4 **return** G plus $(|V| - 2k)$ nodes which are fully connected and are connected to every node in V
5 **else**
6 **return** G plus $2k - |V|$ nodes which have no edges

HALF-CLIQUE



Reduce CLIQUE to HALF-CLIQUE:
Given an instance of CLIQUE, turn it into an instance of HALF-CLIQUE

We're looking for a clique that is bigger than half, so add vertices until k = |V|/2

$f(G, k)$
1 **if** $\lceil |V| \rceil / 2 = k$
2 **return** G
3 **elseif** $k < \lceil |V| \rceil / 2$
4 **return** G plus $(|V| - 2k)$ nodes which are fully connected and are connected to every node in V
5 **else**
6 **return** G plus $2k - |V|$ nodes which have no edges

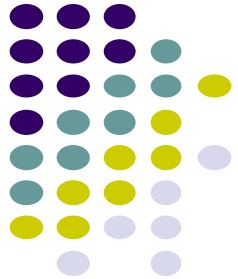
HALF-CLIQUE

Reduce CLIQUE to HALF-CLIQUE:
Given an instance of CLIQUE, turn it into an instance of HALF-CLIQUE

$f(G, k)$
1 **if** $\lceil |V| \rceil / 2 = k$
2 **return** G
3 **elseif** $k < \lceil |V| \rceil / 2$
4 **return** G plus $(|V| - 2k)$ nodes which are fully connected and are connected to every node in V
5 **else**
6 **return** G plus $2k - |V|$ nodes which have no edges

Runtime: From the construction we can see that it is polynomial time

Reduction proof



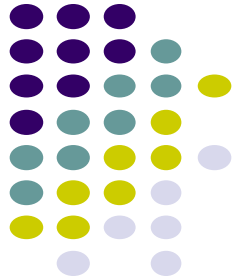
Show that a solution exists to the NP-Complete problem IFF a solution exists *to the NEW problem generate by f*

- Assume we have an NP-Complete problem instance that has a solution, show that the NEW problem instance generated by *f* has a solution

- Assume we have a problem instance of NEW *generated by f* that has a solution, show that we can derive a solution to the NP-Complete problem instance

```
f(G, k)
1  if ⌈|V|⌉/2 = k
2          return G
3  elseif k < ⌈|V|⌉/2
4          return G plus (|V| − 2k) nodes which are fully connected
           and are connected to every node in V
5  else
6          return G plus 2k − |V| nodes which have no edges
```

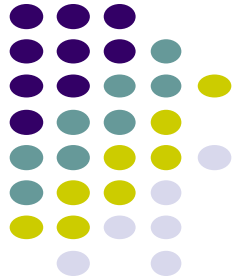
Reduction proof



Given a graph G that has a CLIQUE of size k, show that f(G,k) has a solution to HALF-CLIQUE

If k = |V|/2:

- the graph is unmodified
- f(G,k) has a clique that is half the size



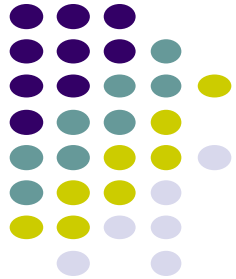
Reduction proof

Given a graph G that has a CLIQUE of size k, show that f(G,k) has a solution to HALF-CLIQUE

If k < |V|/2:

- we added a clique of |V|- 2k fully connected nodes
- there are |V| + |V| - 2k = 2(|V|-k) nodes in f(G)
- there is a clique in the original graph of size k
- plus our added clique of |V|-2k
- k + |V|-2k = |V|-k, which is half the size of f(G)

Reduction proof

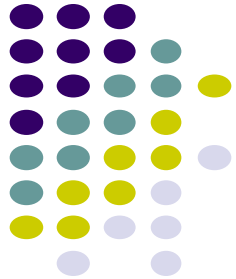


Given a graph G that has a CLIQUE of size k, show that f(G,k) has a solution to HALF-CLIQUE

If k >|V|/2:

- we added 2k - |V| unconnected vertices
- f(G) contains |V| + 2k - |V| = 2k vertices
- Since the original graph had a clique of size k vertices, the new graph will have a half-clique

Reduction proof



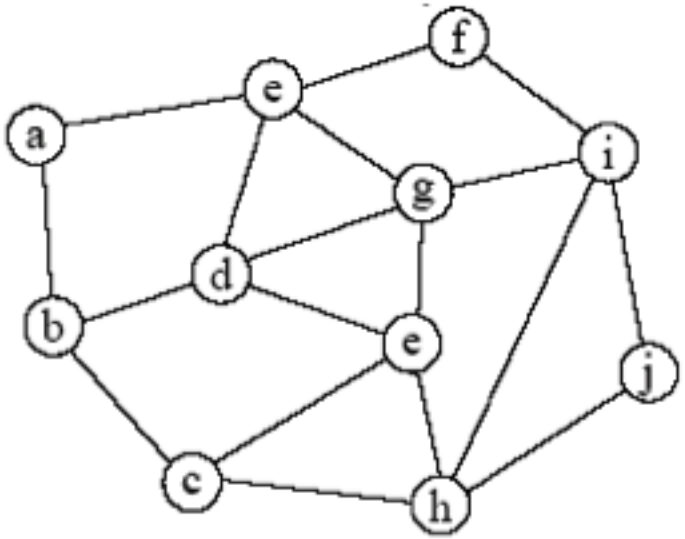
Given a graph f(G) that has a CLIQUE half the elements, show that G has a clique of size k

Key: f(G) was constructed by your reduction function

Use a similar argument to what we used in the other direction

Independent-Set

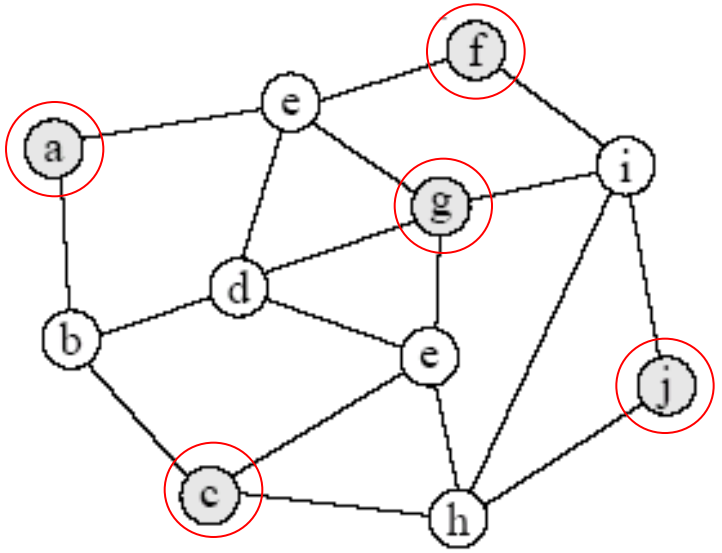
Given a graph G = (V, E) is there a subset V'⊆ V of vertices of size |V'| = k that are independent, i.e. for any pair of vertices u, v ∈ V' there exists no edge between any of these vertices



Does the graph contain an independent set of size 5?

Independent-Set

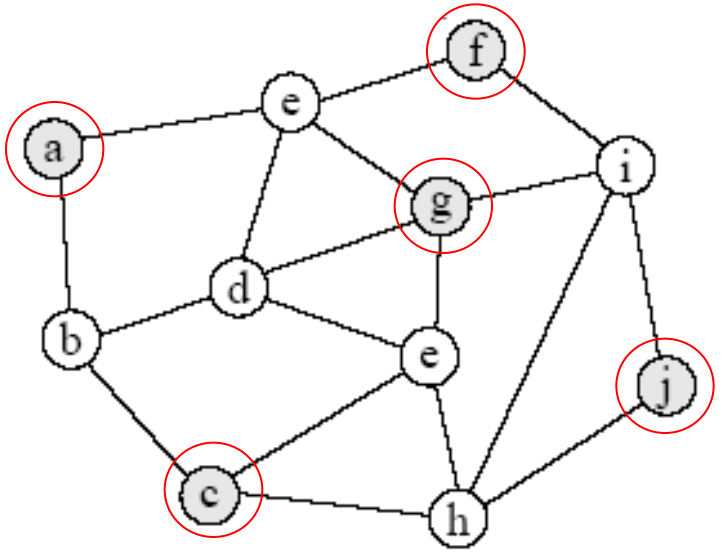
Given a graph G = (V, E) is there a subset V'⊆ V of vertices of size |V '| = k that are independent, i.e. for any pair of vertices u, v ∈ V' there exists no edge between any of these vertices



Independent-Set is NP-Complete

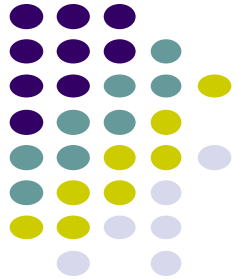
Independent-Set

Given a graph G = (V, E) is there a subset V'⊆ V of vertices of size |V'| = k that are independent, i.e. for any pair of vertices u, v ∈ V' there exists no edge between any of these vertices. Is there an independent set of size k?



Reduce Independent-Set to CLIQUE

Independent-Set to Clique



Given a graph G = (V, E) is there a subset V'⊆ V of vertices of size |V '| = k that are independent, i.e. for any pair of vertices u, v ∈ V' there exists no edge between any of these vertices

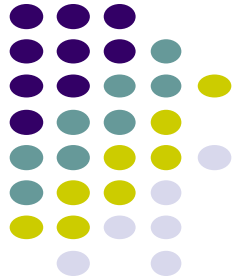
Both are selecting vertices

Independent set wants vertices where NONE are connected

Clique wants vertices where ALL are connected

How can we convert a NONE problem to an ALL problem?

Independent-Set to Clique

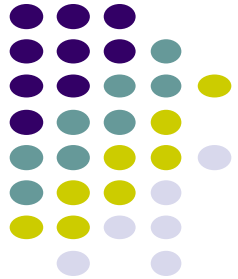


Given a graph G = (V, E), the complement of that graph G' = (V, E) is the a graph constructed by remove all edges E and including all edges not in E

For example, for adjacency matrix this is flipping all of the bits

```
f(G)
    return G'
```

Reduction proof

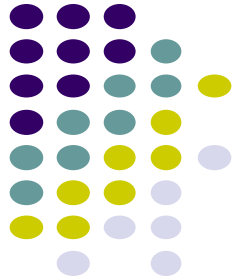


Show that a solution exists to the NP-Complete problem IFF a solution exists *to the NEW problem generate by f*

- Assume we have an Independent-Set problem instance that has a solution, show that the Clique problem instance generated by *f* has a solution
- Assume we have a problem instance of Clique *generated by f* that has a solution, show that we can derive a solution to Independent-Set problem instance

```
f(G)
    return G'
```

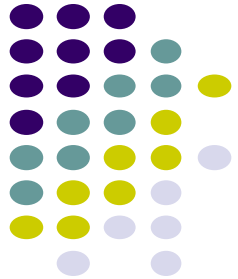
Proof



Given a graph G that has an independent set of size k, show that f(G) has a clique of size k

- By definition, the independent set has no edges between any vertices
- These will all be edges in f(G) and therefore they will form a clique of size k

Proof

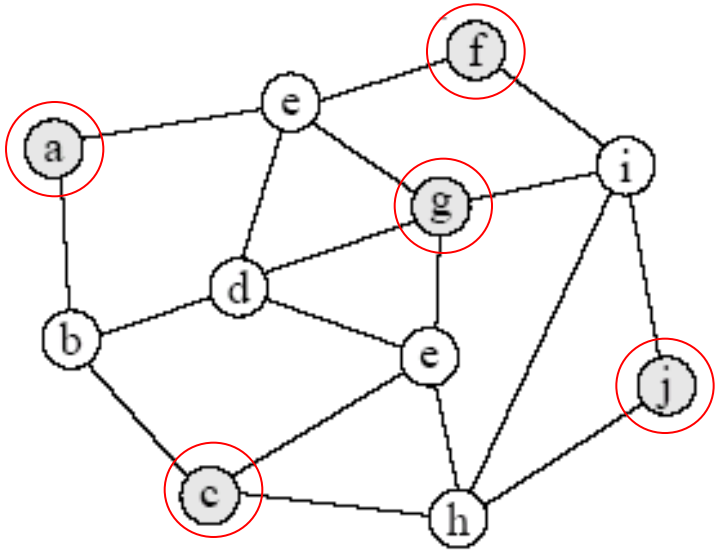


Given f(G) that has clique of size k, show that G has an independent set of size k

- By definition, the clique will have an edge between every vertex
- None of these vertices will therefore be connected in G, so we have an independent set

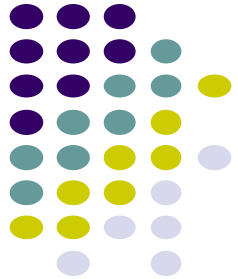
Independent-Set revisited

Given a graph G = (V, E) is there a subset V'⊆ V of vertices of size |V'| = k that are independent, i.e. for any pair of vertices u, v ∈ V' there exists no edge between any of these vertices

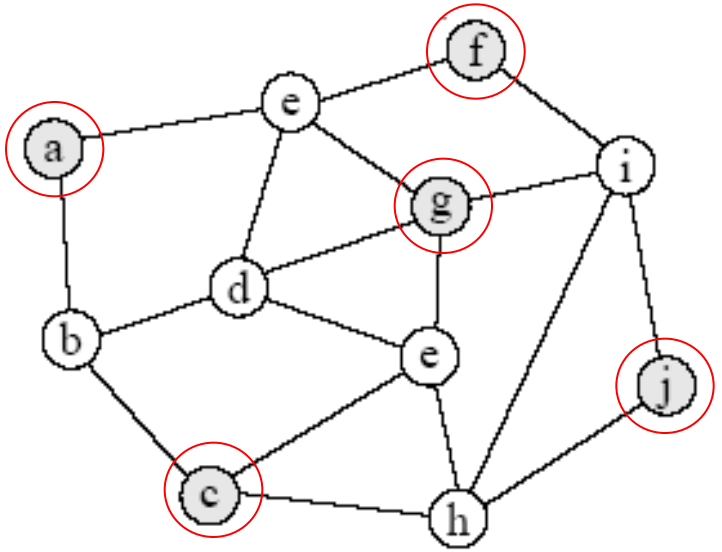


Is Independent-Set NP-Complete?

Independent-Set revisited



Given a graph G = (V, E) is there a subset V'⊆ V of vertices of size |V '| = k that are independent, i.e. for any pair of vertices u, v ∈ V' there exists no edge between any of these vertices



Reduce 3-SAT to Independent-Set

3-SAT to Independent-Set

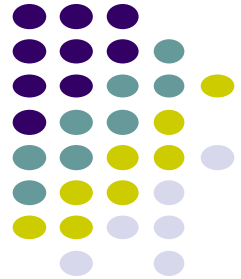
Given a 3-CNF formula, convert it into a graph

$$(a \quad a \quad b)\ (c \quad b \quad d)\ (\ a \quad c \quad d)$$

For the boolean formula in 3-SAT to be satisfied, at least one of the literals in each clause must be true

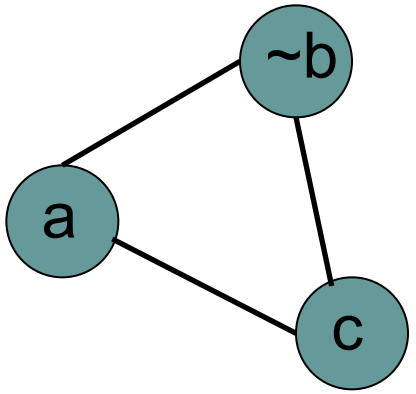
In addition, we must make sure that we enforce a literal and its complement must not both be true.

3-SAT to Independent-Set



Given a 3-CNF formula, convert into a graph

For each clause, e.g. *(a OR not(b) OR c)* create a clique containing vertices representing these literals

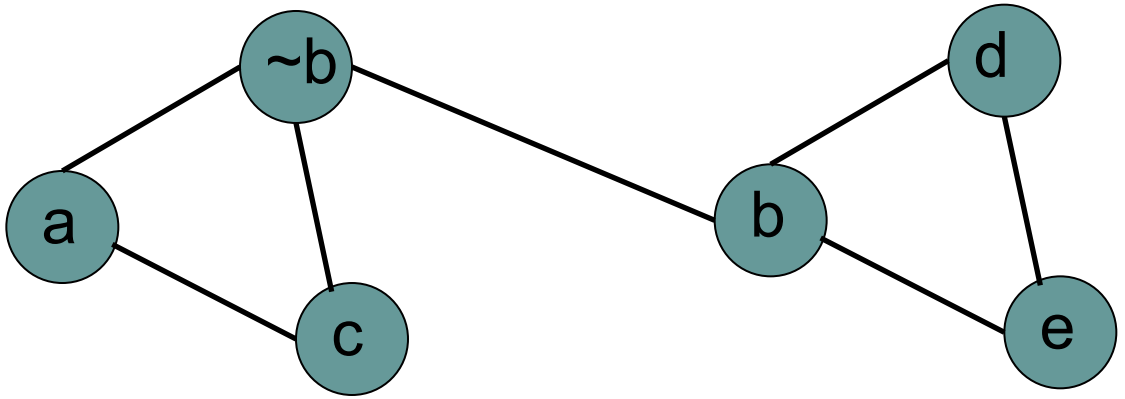


- for the Independent-Set problem to be satisfied it can only select one variable
- to make sure that all clauses are satisfied, we set k = number of clauses

3-SAT to Independent-Set

Given a 3-CNF formula, convert into a graph

To enforce that only one variable and its complement can be set we connect each vertex representing x to each vertex representing its complement ~x



Proof

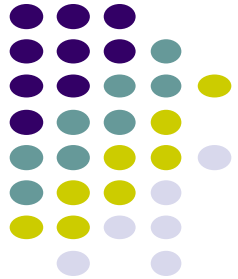
Given a 3-SAT problem with k clauses and a valid truth assignment, show that f(3-SAT) has an independent set of size k. (Assume you know the solution to the 3-SAT problem and show how to get the solution to the independent set problem)

Proof

Given a 3-SAT problem with k clauses and a valid truth assignment, show that f(3-SAT) has an independent set of size k. (Assume you know the solution to the 3-SAT problem and show how to get the solution to the independent set problem)

Since each clause is an OR of variables, at least one of the three must be true for the entire formula to be true. Therefore each 3-clique in the graph will have at least on node that can be selected.

Proof

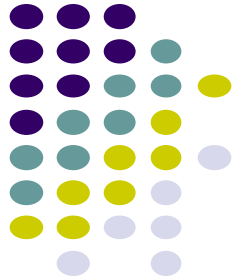


Given a graph with an independent set S of k vertices, show there exists a truth assignment satisfying the boolean formula

- For any variable x_i, S cannot contain both x_i and $\neg x_i$ since they are connected by an edge

- For each vertex in S, we assign it a true value and all others false. Since S has only k vertices, it must have one vertex per clause

More NP-Complete problems



SUBSET-SUM:

- Given a set S of positive integers, is there some subset $S' \subseteq S$ whose elements sum to t.

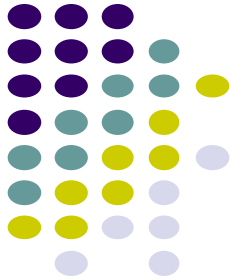
TRAVELING-SALESMAN:

- Given a weighted graph G, does the graph contain a hamiltonian cycle of length k or less?

VERTEX-COVER:

- Given a graph G = (V, E), is there a subset $V' \subseteq V$ such that if $(u,v) \in E$ then $u \in V'$ or $v \in V'$?
- The extra credit was to solve this problem for bipartite graphs

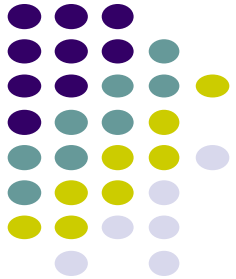
Our known NP-Complete problems



We can reduce any of these problems to a new problem in an NP-completeness proof

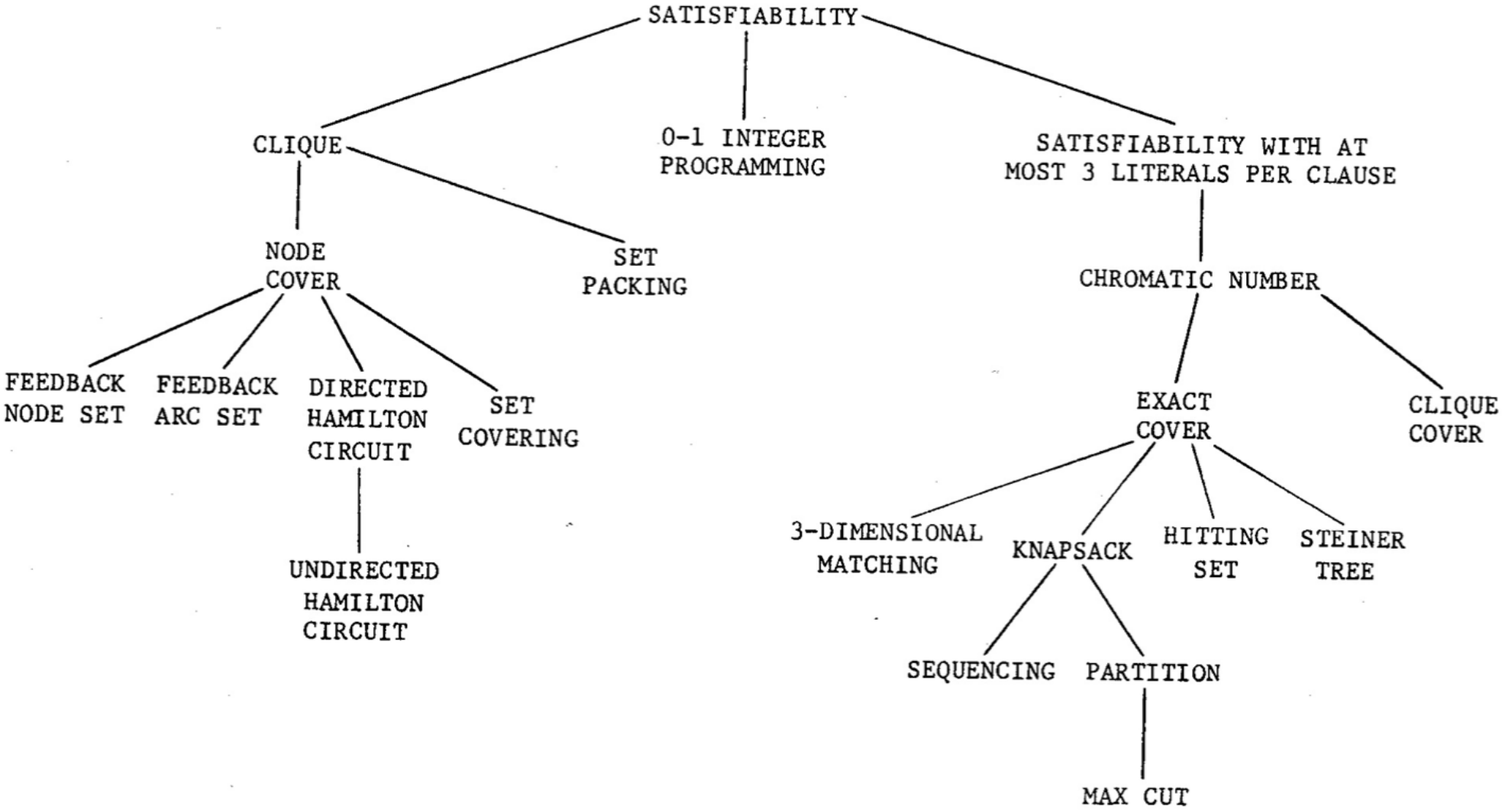
- SAT, 3-SAT
- CLIQUE, HALF-CLIQUE
- INDEPENDENT-SET
- HAMILTONIAN-CYCLE
- TRAVELING-SALESMAN
- VERTEX-COVER
- SUBSET-SUM

The reducibility ‘tree’

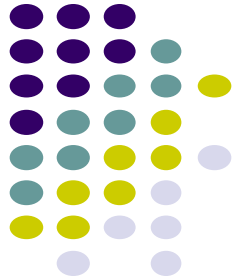


- Richard Karp proved 21 problems to be NP complete in a seminal 1971 paper
- Not that hard to read actually!
- Definitely not hard to read it to the point of knowing what these problems are.
- karp's paper

The reducibility ‘tree’



Search vs. Exists



All the problems we've looked at asked decision questions:

- Is there a hamiltonian cycle?
- Does the graph have a clique of size k?
- Does the graph has an independent set of size k?
- …

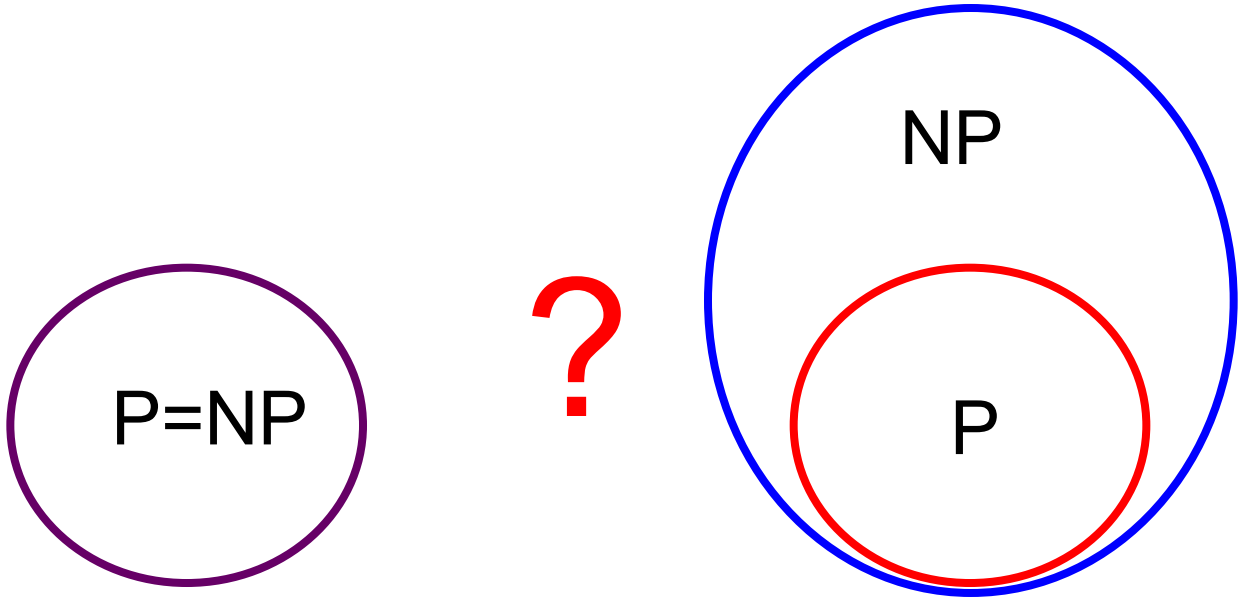
For many of the problems with a k in them, we really want to know what the largest/smallest one is

- What is the largest clique in the graph?
- What is the shortest path that visits all the vertices exactly once?

Why don't we care?

P vs. NP

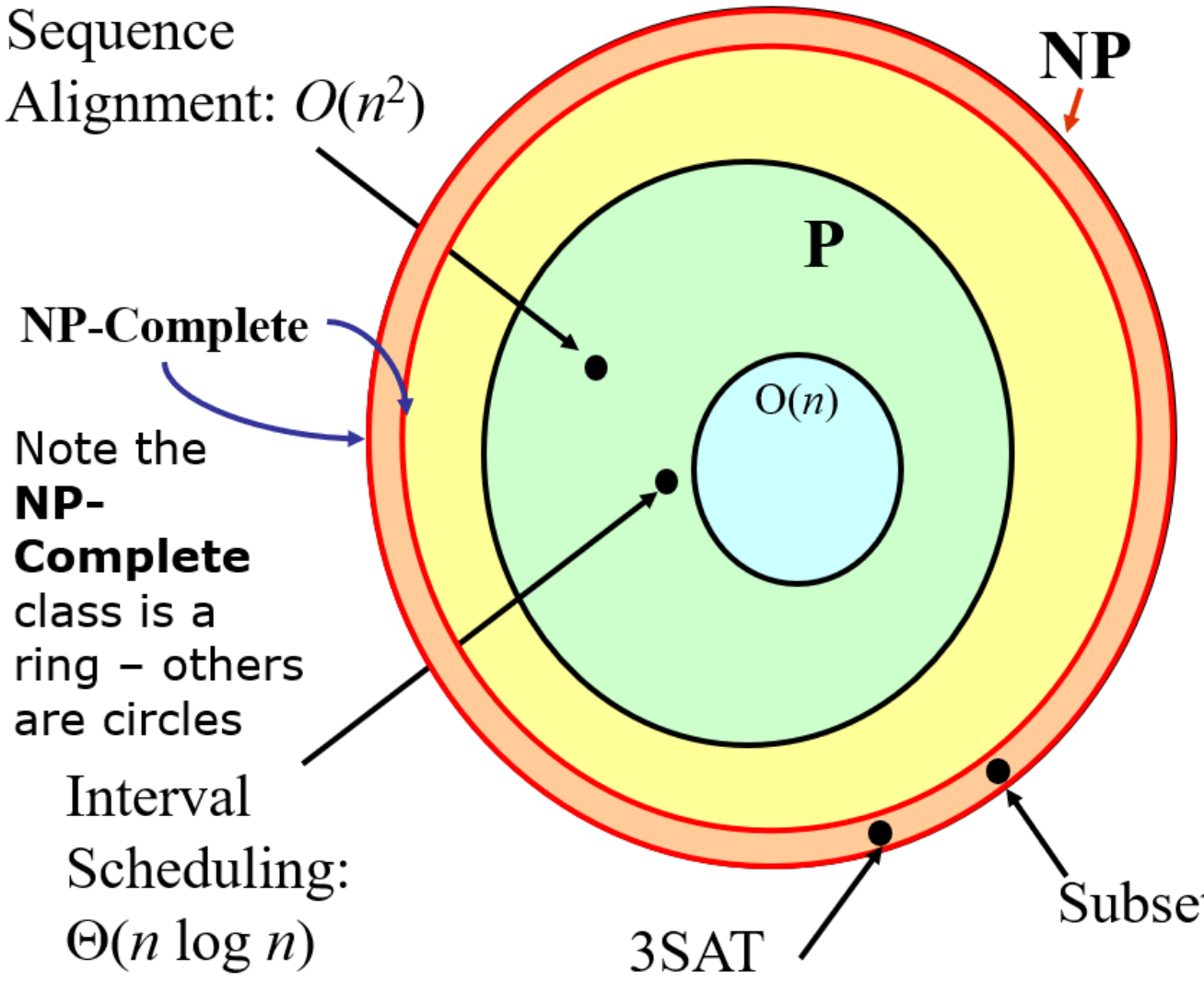
The big question:



Someone finds a polynomial time solution to one of the NP-Complete problems

NP-Complete problems are somehow harder and distinct

Problem Classes if P ≠ NP:



How many problems are in the $\Theta(n)$ class?

infinite

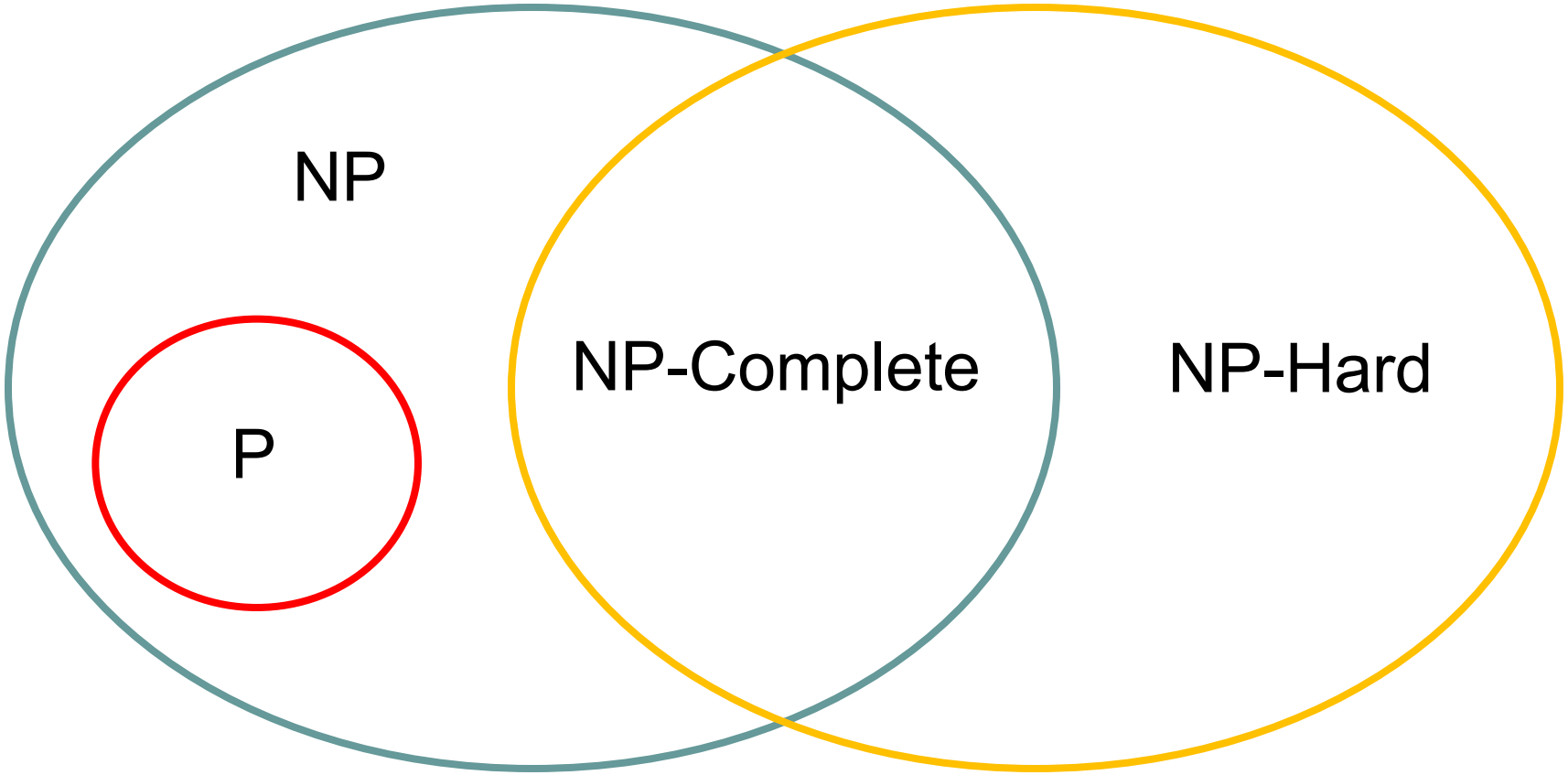
How many problems are in **P** but not in the $\Theta(n)$ class?

infinite

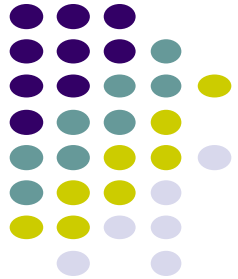
How many problems are in **NP** but not in **P**?

infinite

P vs. NP vs. NP-Complete vs. NP-Hard



P vs. NP vs. NP-Complete vs. NP-Hard



	P	NP	NP-Complete	NP-Hard
Solvable in polynomial time	√			
Solution verifiable in polynomial time	√	√	√	
Reduces any NP problem in polynomial time			√	√

Acknowledgements

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., Introduction to algorithms. MIT press, 2009
- Dr. David Kauchak, Pomona College
- Prof. David Plaisted, The University of North Carolina at Chapel Hill
- Dr. David Evans, University of Virginia